

Chapter 7: Automating Data Preparation Pipelines

<{<http://www.bookeducator.com/Textbook>}learningobjective >Students will be able to design and implement automated data cleaning functions that remove invalid columns, standardize column names, and correct data types

<{<http://www.bookeducator.com/Textbook>}learningobjective >Students will be able to apply mathematical transformations (log, square root, Yeo-Johnson, Box-Cox) to correct skewed distributions in numeric features

<{<http://www.bookeducator.com/Textbook>}learningobjective >Students will be able to implement systematic strategies for handling missing data through imputation or removal based on data characteristics

<{<http://www.bookeducator.com/Textbook>}learningobjective >Students will be able to use AI-assisted coding tools to generate, expand, and generalize data preparation functions

<{<http://www.bookeducator.com/Textbook>}learningobjective >Students will be able to group numeric variables into bins and consolidate rare categorical values to improve data quality

7.1 Introduction



In the previous chapter, we learned how to automate simple univariate statistics and visualizations. Although exploratory data analysis (EDA) often expands quickly into bivariate and multivariate analysis, univariate results almost always surface data quality issues that must be addressed first. For this reason, we intentionally pause deeper EDA here and move ahead to the Data Preparation phase of CRISP-DM.

Data preparation is rarely a single step. It is an iterative, decision-driven process that requires analysts to diagnose data issues, choose appropriate remedies, and validate the results. Historically, this phase has been time-consuming and code-intensive. However, modern AI-assisted coding tools fundamentally change how data preparation work is executed.

In this chapter, you will learn how to design, generate, test, and refine automated data cleaning functions using AI as a coding assistant. Rather than replacing analytical judgment, AI is used to accelerate implementation while keeping responsibility for correctness, assumptions, and validation firmly in the hands of the analyst.

Although real-world datasets vary widely, most data preparation tasks fall into a relatively small set of recurring categories:

- Basic data wrangling
 - Removing unique identifiers and non-analytical columns
 - Eliminating empty, constant, or structurally invalid columns
 - Standardizing column names and data types
 - Standardizing inconsistent categorical group values (e.g., changing "Utah", "UTAH", "UT", "ut" all to a single value so they are treated the same)
- Dates and times
 - Parsing and validating date and time fields
 - Handling time zones and inconsistent formats
 - Extracting useful temporal features such as year, month, or day of week
- Grouping and transformation
 - Grouping numeric variables into bins or categories
 - Applying mathematical transformations to improve interpretability or modeling performance
- Handling missing data
 - Dropping rows or columns with excessive missing values

- Imputing missing values using simple or model-based approaches
- Managing outliers
 - Identifying numeric and categorical outliers using defined rules or thresholds
 - Addressing outliers through removal, capping, transformation, or reclassification

Each section in this chapter focuses on one of these categories. You will first review the underlying concept and common strategies, then work through an AI-assisted function generation exercise. For each task, you will be given a baseline Python-specific prompt designed to produce a reusable data cleaning function.

You will be asked to modify the prompt, adapt it to your dataset, run the generated code, test its behavior, and save the function for reuse. This process mirrors modern analytics workflows, where analysts increasingly act as designers, reviewers, and validators of automated solutions rather than writing every line of code from scratch.

By the end of this chapter, you will have a library of tested, modular data preparation functions and the skills needed to responsibly use AI-assisted coding as part of the CRISP-DM Data Preparation phase.

7.2 Data Wrangling

This video can be viewed [online](#).

Data wrangling, also known as data munging, is the process of cleaning, transforming, and structuring raw data into a usable format for analysis. This

process includes removing irrelevant columns, correcting structural issues, standardizing formats, and preparing data so that downstream analysis and modeling behave as expected.

In this section, we begin by performing several foundational data wrangling steps without the help of AI. This is intentional. Before relying on AI-assisted code generation, it is essential to understand what these operations do, why they are needed, and how they can be implemented manually.

These first steps establish a baseline function that performs a small but meaningful set of automated wrangling tasks. In the second half of this section, we will use this baseline as an input to an AI agent and ask it to expand, optimize, and generalize the function in ways that would be time-consuming to implement by hand.

To get started, we will load several datasets with different wrangling challenges so we can evaluate how well our function generalizes across data types.

```
# Load sample datasets

import pandas as pd

# Datasets with numeric labels for testing
df_insurance = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/data/insurance.csv')
df_nba = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/data/nba_salaries.csv')
df_airbnb = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/data/listings.csv')

# Dataset with categorical labels for testing
df_airline = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/data/airline_satisfaction.csv')
```

First Steps: Building a Basic Wrangling Function

To keep our scope manageable, we will focus on a small set of common wrangling tasks that frequently appear at the beginning of the Data

Preparation phase:

1. Remove empty columns or those with more than 95% missing values
2. Remove columns with all unique values or those with more than 95% unique values
3. Remove columns containing only a single repeated value

Rather than building this function step by step as we did in the previous chapter, a complete solution is provided below. Before reviewing it, you are encouraged to attempt your own implementation. Even a partial solution will help you better understand the design decisions in the function that follows.

```
# Function for basic data wrangling
# Required parameter: a Pandas DataFrame
# Optional parameters:
#   features = list of column names to process
#   missing_threshold = percentage of missing values required to drop a column
#   unique_threshold = percentage of unique values in a column before it's removed
#   messages = boolean flag to print messages when columns are dropped

def basic_wrangling(df, features=[], missing_threshold=0.95, unique_threshold=0.95,
messages=True):
    import pandas as pd

    if not features:
        features = df.columns

    for feat in features:
        if feat in df.columns:
            missing = df[feat].isna().sum()
            unique = df[feat].nunique()
            rows = df.shape[0]

            if missing / rows >= missing_threshold:
                if messages: print(f"Dropping {feat}: {missing} missing values out of
{rows} ({round(missing/rows, 2)})")
                df.drop(columns=[feat], inplace=True)
            elif unique / rows >= unique_threshold:
                if df[feat].dtype in ['int64', 'object']:
                    if messages: print(f"Dropping {feat}: {unique} unique values out of
{rows} ({round(unique/rows, 2)})")
                    df.drop(columns=[feat], inplace=True)
                elif unique == 1:
                    if messages: print(f"Dropping {feat}: Contains only one unique value
({df[feat].unique()[0]})")
                    df.drop(columns=[feat], inplace=True)
            else:
                if messages: print(f"Skipping {feat}: Column not found in
DataFrame")

    return df
```

This function evaluates each column against simple structural rules and removes features that are unlikely to be analytically useful. While limited in scope, it demonstrates several important design patterns, including configurable thresholds, optional messaging, and reusable function structure.

AI-Assisted Function Generation: Expanding the Wrangling Pipeline

The function above represents a reasonable first draft, but real-world data wrangling often requires far more flexibility, efficiency, and transparency. This is where AI-assisted coding becomes especially valuable.

Your goal in this next step is not to discard the existing function, but to treat it as a specification artifact. You will provide it to an AI coding assistant along with a detailed prompt and ask the agent to produce an enhanced, production-ready wrangling function.

Below is a baseline Python-specific prompt you should use as a starting point. You are expected to modify and extend this prompt based on your dataset, preferences, and analytical goals.

Baseline AI Prompt:

You are an expert Python data engineer. Given the following Pandas function that performs basic data wrangling, redesign and expand it into a robust, efficient, and reusable data wrangling function. Preserve the original behavior, but improve structure, efficiency, and extensibility. Add support for column name standardization, optional duplicate row handling, configurable logging instead of print statements, clear docstrings, type hints, and safe handling of edge cases. The function should not mutate the input DataFrame unless explicitly specified. Use vectorized Pandas operations where possible and include inline comments explaining major design decisions.

After generating the function, review the code carefully. Test it on multiple datasets, compare its output to the original function, and make any necessary adjustments. Save the final version for reuse later in the chapter and in the course assignment.

This process mirrors modern analytics workflows, where analysts are responsible for specifying requirements, validating outputs, and ensuring that automated solutions behave correctly and transparently.

7.3 Dates and Times



This video can be viewed [online](#).

Dates and times frequently appear early in the Data Preparation phase and often require special handling. Raw date values are typically stored as strings and cannot be used directly in analytical models. Treating dates as categorical values is rarely appropriate because most date fields contain too many unique values to satisfy minimum category frequency requirements.

A common strategy is to decompose date and time values into structured numeric or standardized categorical features such as year, month, day of month, weekday, hour, or minute. These derived features preserve temporal information in a form that can be meaningfully used in analysis and modeling.

Another common approach is to compute time spans. For example, analysts may calculate the number of days since a customer's last purchase, the duration of employment at the time of evaluation, or the time between order placement and fulfillment. These elapsed-time features often carry more predictive signal than raw dates.

As with data wrangling, we begin by implementing a simple date parsing function without the help of AI. This ensures you understand the mechanics of date conversion, feature extraction, and basic error handling before using AI to extend and generalize the solution.

First Steps: Parsing Date and Time Features

The function below demonstrates a basic approach to parsing date columns and extracting commonly used features.

```
def parse_date(df, features=[], days_since_today=False, drop_date=True, messages=True):
    import pandas as pd
    from datetime import datetime as dt

    for feat in features:
        if feat in df.columns:
            df[feat] = pd.to_datetime(df[feat])
            df[f'{feat}_year'] = df[feat].dt.year
```

```

df[f'{feat}_month'] = df[feat].dt.month
df[f'{feat}_day'] = df[feat].dt.day
df[f'{feat}_weekday'] = df[feat].dt.day_name()

if days_since_today:
    df[f'{feat}_days_until_today'] = (dt.today() - df[feat]).dt.days
if drop_date:
    df.drop(columns=[feat], inplace=True)
else:
    if messages:
        print(f'{feat} does not exist in the DataFrame provided. No work performed.')

return df

```

This function accepts a DataFrame and a list of column names expected to contain date values. For each valid column, it converts the data to a datetime format, extracts multiple time-based features, and optionally computes the number of days between the date and today. Basic error handling ensures that missing columns do not cause the function to fail.

To test the function, we apply it to the Airbnb dataset using one valid date column and one invalid column name to confirm that the function handles errors gracefully.

```

df_airbnb = parse_date(df_airbnb, days_since_today=True, features=['last_review',
'doesnt exist'])
df_airbnb.head()

# Output:
# See the output in your own code

```

After running the function, scroll to the far right of the DataFrame to locate the newly created features, including year, month, day, weekday, and days since today.

AI-Assisted Function Generation: Robust Date and Time Handling

The function above demonstrates core concepts but makes several simplifying assumptions. Real-world datasets often include inconsistent formats, invalid

dates, time zones, missing values, and mixed date-time representations. AI-assisted coding can significantly reduce the effort required to handle these complexities.

In this step, treat the existing function as a baseline specification and provide it to an AI coding assistant. Your goal is to generate a more robust, flexible, and production-ready date parsing function.

Baseline AI Prompt:

You are an expert Python data engineer. Given the following Pandas function that parses date and time features, redesign and expand it into a robust and reusable date handling function. Preserve the original behavior, but add support for multiple date formats, optional time zone handling, safe coercion of invalid dates, configurable feature extraction (year, month, weekday, hour, etc.), clear docstrings, type hints, and configurable logging. The function should avoid mutating the input DataFrame unless explicitly requested and should handle missing or malformed values gracefully.

After generating the function, carefully review the code, test it on multiple datasets, and compare its output to the original implementation. Modify the prompt or the generated code as needed, then save the final function for reuse in later sections and in the chapter assignment.

7.4 Grouping Data into Bins



Grouping data into bins is a technique used in data preprocessing where continuous or categorical values are divided into discrete groups. This process, also called **binning** or **discretization**, simplifies analysis by converting raw values into meaningful categories. However, binning categorical data versus numeric data occurs for different reasons and requires different decision rules.

Categorical Binning

This video can be viewed [online](#).

Categorical binning is useful for several common reasons:

- **There are too many unique categories:** High-cardinality categorical features can overwhelm models and reduce interpretability.
- **Some categories are too rare:** Categories that appear in very few records often provide little analytical value.
- **You want to combine similar groups:** Functionally similar categories can often be grouped to simplify analysis.

We use the **5% rule** A common guideline in data preprocessing stating that each category should represent at least five percent of the dataset to support meaningful analysis. to evaluate categorical group sizes. If a group represents less than 5% of the data, it may be too small to provide reliable insight. This rule is best treated as a diagnostic guideline rather than a strict requirement, especially when working with large datasets.

When a category violates the 5% rule, analysts typically choose one of three actions:

- Merge it with other similar categories.
- Group it into an “Other” category, provided the combined group now exceeds the threshold.
- Remove the records from the dataset.

This rule helps prevent models from overfitting to rare categories. It also keeps datasets more manageable and interpretable. Let’s review an example and then automate the binning process.

Table 7.1
Transaction Dataset

Payment Method	Count	Percentage
Credit Card	600	60%
PayPal	200	20%

Payment Method	Count	Percentage
Apple Pay	90	9%
Google Pay	60	6%
Bitcoin	30	3%
Cash	20	2%

Because Bitcoin and Cash each represent less than 5% of the dataset, we should either remove them, merge them with other categories, or group them into “Other.” Since they do not have obvious matches and together represent exactly 5%, grouping them into “Other” is a reasonable choice.

The function below automates this process:

```
# Documentation:
# required: a Pandas DataFrame
# optional:
# feature = the name of the feature that needs to be binned; default = 'all' meaning
all features will be binned
# cutoff = the minimum percent of rows required to represent a group value without
being binned into 'Other'
# replace_with = the group name assigned to the 'Other' category
# NOTES: this function will only apply to categorical values

def bin_categories(df, features=[], cutoff=0.05, replace_with='Other', messages=True):
    import pandas as pd

    for feat in features:
        if feat in df.columns:
            if not pd.api.types.is_numeric_dtype(df[feat]):
                other_list = df[feat].value_counts()[df[feat].value_counts() / df.shape[0] <=
cutoff].index
                df.loc[df[feat].isin(other_list), feat] = replace_with
            else:
                if messages:
                    print(f'{feat} not found in the DataFrame provided. No binning performed')

    return df
```

This function bins low-frequency categorical values based on a configurable cutoff. Because it checks data types, it can safely be applied to all columns without affecting numeric features.

```
df_airbnb = basic_wrangling(df_airbnb, features=df_airbnb.columns)
df_airbnb = bin_categories(df_airbnb, features=df_airbnb.columns)
```

```
df_airbnb.head()

# Output:
# Too many unique values (20025 out of 20025, 1.0) for id
# Too many unique values (19542 out of 20025, 1.0) for name
# Too much missing (20025 out of 20025, 1.0) for neighbourhood_group

# See DataFrame head() in your own notebook.
```

After binning, “Other” becomes the largest category. This indicates that many groups fell below the cutoff. This function implements only one of several valid strategies, so domain knowledge should guide whether a different approach is more appropriate.

AI-Assisted Function Generation: Categorical Binning

The function above provides a solid baseline, but real-world categorical binning often requires more flexibility and transparency.

Baseline AI Prompt:

You are an expert Python data engineer. Given the following Pandas function that bins low-frequency categorical values, redesign and expand it into a robust, reusable categorical binning function. Preserve the original behavior, but add support for automatic feature selection, configurable minimum group size rules, optional removal instead of relabeling, logging instead of print statements, clear docstrings, type hints, and safe handling of missing values. The function should avoid mutating the input DataFrame unless explicitly requested and should return both the transformed DataFrame and a summary of binning actions performed.

Numeric Binning

Numeric binning converts continuous values into discrete ranges. It is commonly used to reduce noise, improve interpretability, or support algorithms that perform better with categorical inputs.

- **Reduce noise and variability:** Binning can simplify skewed or irregular distributions.
- **Improve interpretability:** Broad categories are often easier to reason about than raw numeric values.
- **Support specific algorithms:** Some models, such as decision trees, naturally work with categorical inputs.

```
# Sample Data
data = {'Age': [5, 12, 25, 32, 45, 51, 63, 77]}
df = pd.DataFrame(data)

bins = [0, 18, 35, 50, 100]
labels = ['Child', 'Young Adult', 'Middle-aged', 'Senior']

df['Age Group'] = pd.cut(df['Age'], bins=bins, labels=labels)
print(df)

# Output:
# See the output in your own code
```

AI-Assisted Function Generation: Numeric Binning

Baseline AI Prompt:

You are an expert Python data engineer. Design a reusable numeric binning function for Pandas DataFrames that supports both fixed-threshold and quantile-based binning. The function should allow configurable bin edges, labels, handling of missing values, optional outlier clipping, clear documentation, type hints, and a summary of binning decisions. Preserve row counts and avoid mutating the input DataFrame unless explicitly requested.

With these techniques in place, we can now move on to additional forms of automated data transformation.

7.5 Math Transformations

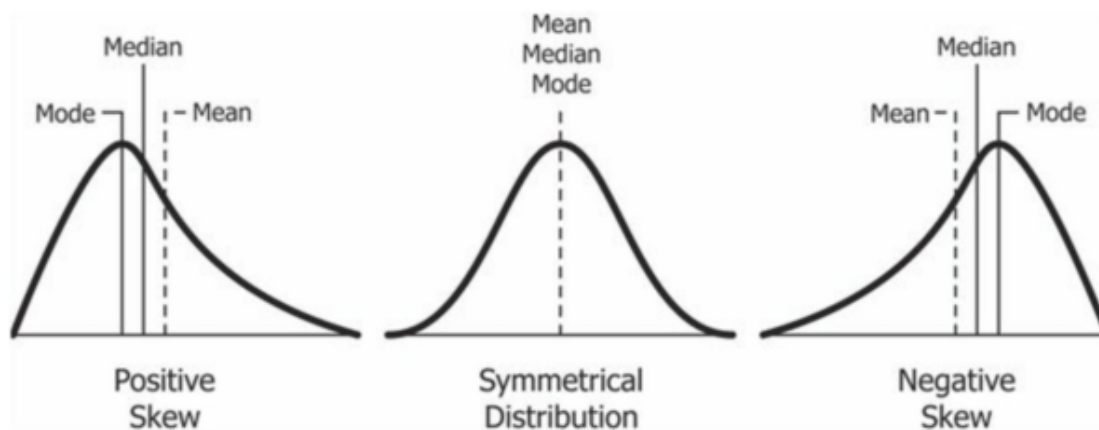


Figure 7.1: Representation of Skewness

Photo by Diva Jain, August 23, 2018, [CC BY-SA 4.0](#) via [Wikimedia Commons](#).

This video can be viewed [online](#).

An Adjustment Too Far

In the video above, the transformation process goes a bit too far by optimizing the skew correction at an overly granular level. While this level of tuning can be useful later in the modeling stage—particularly for interpreting coefficients and making statistical inferences—it can be counterproductive for prediction by introducing unnecessary complexity and potential overfitting. For this reason, the example code presented below intentionally omits that level of fine-grained optimization.

Why Do We Transform?

Mathematical transformations are often applied to numeric features in data analytics to improve the performance of machine learning models, enhance

interpretability, and correct distributional issues. Below are some key reasons why transformations are beneficial:

- **Normalizing Skewed Data:** Many machine learning algorithms assume normality in the input data. If a feature is highly skewed (for example, income), applying a transformation can help make the distribution more symmetric.
- **Reducing the Impact of Outliers:** Outliers can disproportionately affect models like linear regression. Logarithmic or root transformations can reduce their influence.
- **Improving Linearity:** Some machine learning models perform better when relationships between features and the target variable are closer to linear. Transformations can help accomplish this.
- **Stabilizing Variance:** If a feature’s variance increases as its value increases, a transformation can help achieve homoscedasticity (constant variance), which is desirable in linear regression.
- **Making Data More Interpretable:** Some transformations help present data in a way that is easier to understand, such as converting exponential growth to a more linear scale.

The list above explains why transformations can be useful. The next obvious question is how to select a reasonable transformation and how to automate that decision.

How Do We Transform?

The table below summarizes several common transformations:

Table 7.2
Summary of Common Transformations

Transformation	Use Case	Effect	Reverse
Log	Right-skewed data (e.g., prices, income)	Reduces skewness, stabilizes variance	Exponentiation (<code>np.exp(x)</code>)
Square Root (including n root)	Count data (e.g., population, transaction counts)	Reduces skewness while preserving order	Power (x^{**2})
Yeo-Johnson	Skewed data that may include zeros or negative values	Automatically chooses a power-style transform that often improves normality without requiring strictly positive values	Inverse Yeo-Johnson (requires the fitted lambda; typically handled via libraries that store parameters)
Box-Cox	Non-normal data (strictly positive)	Stabilizes variance and improves normality	Inverse Box-Cox (<code>scipy.special.inv_boxcox(x, lambda)</code>)
Min-Max Scaling	Features with different units	Scales features to [0,1] range	$x * (\max - \min) + \min$

Transformation	Use Case	Effect	Reverse
Z-Score Standardization	Features with different scales	Centers data around 0 with unit variance	$x * \text{std} + \text{mean}$

You will not likely fully understand why these transformations matter until the modeling phase. For now, the goal is to learn how to apply them safely and to automate a reasonable choice when a feature is clearly skewed.

One practical approach is to try a small, interpretable menu of monotonic transformations (such as log, square root, cube root, and Yeo-Johnson) and then select the option that makes the skewness closest to zero. This avoids excessive tuning and keeps the results understandable and repeatable.

Yeo-Johnson Transformation (in Plain English)

The Yeo-Johnson transformation is a power-based technique that is similar in spirit to Box-Cox, but it works even when the data contains zeros and negative values. In practice, the method searches for a transformation strength (a parameter often called lambda) that makes the transformed values closer to a normal distribution.

The key advantage is safety and convenience: rather than manually shifting data to be positive before applying a log transform, Yeo-Johnson can handle a wider range of real-world numeric messiness while still preserving rank order (it is monotonic for the fitted parameter). This makes it a strong default option when you want an automated approach.

Let's begin by examining the histogram of a skewed feature and see how various transformations affect the shape. Remember that we have four

datasets to test our functions on. We will use the *charges* feature from the medical insurance dataset:



```
# Mount Google Drive if needed and bring in some sample data
from google.colab import drive
drive.mount('/content/drive')

import pandas as pd

# Datasets with numeric label for testing
df_insurance = pd.read_csv('/content/drive/MyDrive/Colab
Notebooks/data/insurance.csv')
df_nba = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/data/nba_salaries.csv')
df_airbnb = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/data/listings.csv')

# Dataset with categorical label for testing
df_airline = pd.read_csv('/content/drive/MyDrive/Colab
Notebooks/data/airline_satisfaction.csv')

import seaborn as sns, matplotlib.pyplot as plt, numpy as np

print(f"Original charges: {df_insurance['charges'].skew()}")
print(f"Square root transform: {(df_insurance['charges']**(1/2)).skew()}")
print(f"Cubed root transform: {(df_insurance['charges']**(1/3)).skew()}")
print(f"Log transform (log2): {np.log2(df_insurance['charges']).skew()}")

# Create subplots (1 row, 4 columns)
fig, axes = plt.subplots(1, 4, figsize=(20, 5))

# Original charges histogram
sns.histplot(data=df_insurance, x='charges', ax=axes[0])
axes[0].set_title("Original Distribution")

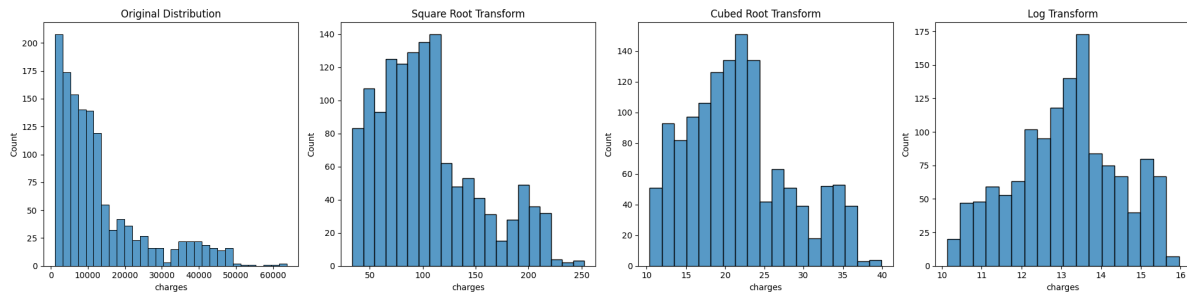
# Square root transform
sns.histplot(df_insurance['charges']**(1/2), ax=axes[1])
axes[1].set_title("Square Root Transform")

# Cubed root transform
sns.histplot(df_insurance['charges']**(1/3), ax=axes[2])
axes[2].set_title("Cubed Root Transform")

# Log transform
sns.histplot(np.log2(df_insurance['charges']), ax=axes[3])
axes[3].set_title("Log Transform")

# Adjust layout and show plots
plt.tight_layout()
plt.show()

# Output:
# Original charges:      1.5158796580240388
# Square root transform:  0.7958625166976426
# Cubed root transform:   0.515182615434519
# Log transform (log2):  -0.09009752473024946
```



Notice how the skewness and shape of the histogram move closer to a normal distribution as we apply stronger transformations. However, stronger transformations are not automatically better. A transformation can over-correct and introduce skew in the opposite direction. In practice, we typically try a small menu of transformations and choose the one that brings skewness closest to zero while keeping the result interpretable.

The automated function below removes the overly granular exponent tuning from the earlier example and replaces it with a menu-based approach that is easier to explain, faster to run, and more stable across different samples.



```
def skew_correct(df, feature, methods=None, messages=True, visualize=True):
    import pandas as pd, numpy as np
    import seaborn as sns, matplotlib.pyplot as plt

    # Default menu of options (simple, interpretable, and fast)
    if methods is None:
        methods = ["none", "cbrt", "sqrt", "log1p", "yeojohnson"]

    # Ensure the feature exists
    if feature not in df.columns:
        if messages:
            print(f"{feature} is not found in the DataFrame. No transformation performed")
        return df

    # Coerce to numeric (non-numeric becomes NaN)
    x = pd.to_numeric(df[feature], errors="coerce")
    if x.notna().sum() == 0:
        if messages:
            print(f"{feature} could not be converted to numeric values. No transformation performed")
        return df
```

```

out = df.copy()

# Helper: shift negatives for transforms that require non-negative values
def _shift_nonneg(s: pd.Series):
    min_val = s.min(skipna=True)
    if pd.isna(min_val):
        return s, 0.0
    shift = -float(min_val) if min_val < 0 else 0.0
    return s + shift, shift

x_shifted, shift_amt = _shift_nonneg(x)

candidates = {}

# Candidate: none
candidates["none"] = x.astype("float64")

# Cube root (monotonic; using shifted non-negative for consistency)
candidates["cbt"] =
np.cbrt(x_shifted.clip(lower=0)).astype("float64")

# Square root (requires non-negative)
candidates["srt"] =
np.sqrt(x_shifted.clip(lower=0)).astype("float64")

# Log1p (requires non-negative)
candidates["log1p"] =
np.log1p(x_shifted.clip(lower=0)).astype("float64")

# Yeo-Johnson (works with negatives without shifting), requires scipy
if "yeojohnson" in methods:
    try:
        from scipy.stats import yeojohnson

        # Transform only non-missing values; yeojohnson returns float array + lambda
        x_nonmissing = x.dropna().to_numpy(dtype="float64")
        yj_vals, yj_lambda = yeojohnson(x_nonmissing)

        # CRITICAL FIX:
        # Ensure destination is float BEFORE assigning float transformed values
        yj_series = x.astype("float64").copy()
        yj_series.loc[x.dropna().index] = yj_vals

        candidates["yeojohnson"] = yj_series
    except Exception:
        if messages:
            print("scipy not available (or Yeo-Johnson failed). Skipping
yeojohnson.")

# Evaluate candidates by closeness of skewness to zero
best_name = None
best_series = None
best_score = np.inf

for name in methods:
    if name not in candidates:
        continue
    sk = candidates[name].skew(skipna=True)
    score = abs(sk) if not pd.isna(sk) else np.inf
    if score < best_score:
        best_score = score
        best_name = name
        best_series = candidates[name]

```

```

new_col = f"{feature}_skewfix"
out[new_col] = best_series.astype("float64")

if messages:
    before = x.skew(skipna=True)
    after = out[new_col].skew(skipna=True)
    print(f"Feature: {feature}")
    print(f"Skew before: {round(before, 5)}")
    print(f"Chosen method: {best_name}")
    if best_name in ["cbrt", "sqrt", "log1p"]:
        print(f"Shift used (to handle negatives): {round(shift_amt, 5)}")
    print(f"Skew after: {round(after, 5)}")
    print(f"New column: {new_col}")

# Optional visualization
if visualize:
    df_temp = pd.DataFrame({feature: x.astype("float64"),
    "transformed": out[new_col].astype("float64")})
    f, axes = plt.subplots(1, 2, figsize=[7, 3.5])
    sns.despine(left=True)
    sns.histplot(df_temp[feature].dropna(), ax=axes[0], kde=True)
    axes[0].set_title("Before")
    sns.histplot(df_temp["transformed"].dropna(), ax=axes[1], kde=True)
    axes[1].set_title("After")
    plt.setp(axes, yticks=[])
    plt.tight_layout()
    plt.show()

return out

```

This menu-based function is still a meaningful automation exercise, but it is easier to understand than finely tuning a power parameter. If you feel uncertain about any step, follow along with the video above and test the function on several features across different datasets.

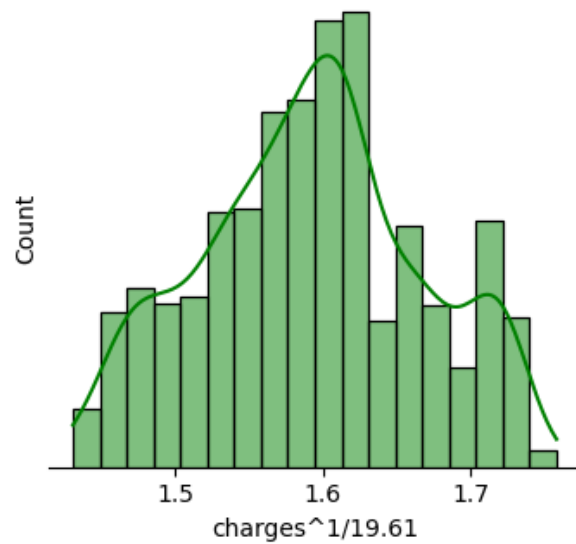
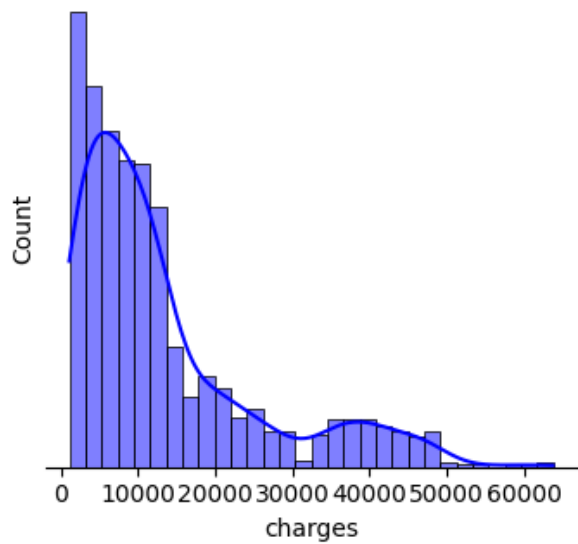


```

import pandas as pd
df_insurance = pd.read_csv('/content/drive/MyDrive/Colab
Notebooks/data/insurance.csv')
skew_correct(df_insurance, 'charges').head()

# Output:
# Feature: charges
# Skew before: 1.51588
# Chosen method: yeojohnson
# Skew after: -0.00871
# New column: charges_skewfix

```



	age	sex	bmi	children	smoker	region	charges	charges ^{1/19.61}
0	19	female	27.900	0	yes	southwest	16884.92400	1.642777
1	18	male	33.770	1	no	southeast	1725.55230	1.462397
2	28	male	33.000	3	no	southeast	4449.46200	1.534770
3	33	male	22.705	0	no	northwest	21984.47061	1.665036
4	32	male	28.880	0	no	northwest	3866.85520	1.523825

You might find it beneficial to try this out on a few other datasets:



```
df_nba = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/data/nba_salaries.csv')
skew_correct(df_nba, 'Salary').head()

# Output:
# Feature: Salary
# Skew before: 1.84165
# Chosen method: yeojohnson
# Skew after: -0.02333
# New column: Salary_skewfix
```



```
df_airbnb = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/data/listings.csv')
skew_correct(df_airbnb, 'average_review').head()
```

```
# Output:
# Feature: average_review
# Skew before: 7.58989
# Chosen method: yeojohnson
# Skew after: 0.23334
# New column: average_review_skewfix
```



```
df_airbnb = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/data/listings.csv')
skew_correct(df_airbnb, 'average_review').head()
```

```
# Output:
# Feature: Departure Delay in Minutes
# Skew before: 6.82198
# Chosen method: yeojohnson
# Skew after: 0.49398
# New column: Departure Delay in Minutes_skewfix
```

As you can see the Yeo-Johnson technique is pretty powerful and a nice goto option. This function is only the beginning of automating mathematical operations. As you learn more in the modeling phase, you will discover additional transformations and evaluation strategies that can be valuable in specific contexts.

AI-Assisted Function Generation: Improving Mathematical Transformations

At this point, you have seen how a thoughtfully designed transformation function can significantly reduce skewness while remaining interpretable and robust. However, real-world data pipelines often require even greater flexibility, scalability, and transparency than what a single hand-crafted function can reasonably provide.

Rather than extending this function manually, this is an ideal opportunity to leverage AI-assisted coding. Your goal is not to replace your understanding of

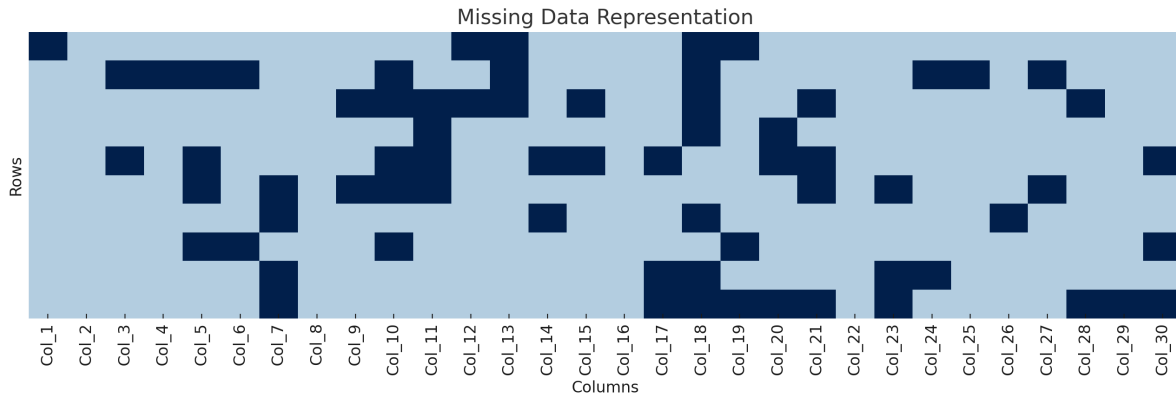
transformations, but to use AI as a collaborator that helps generalize, harden, and document your work.

Baseline AI Prompt:

You are an expert Python data engineer. Given the following Pandas function that performs skewness reduction using a limited set of monotonic mathematical transformations, redesign and generalize it into a reusable, production-quality utility. Preserve the core behavior and intent of the original function, but improve it by adding support for multiple selectable transformation options (e.g., log1p, square root, cube root, Yeo-Johnson), explicit skewness thresholds for deciding whether to transform, safe handling of zeros and negative values, clear docstrings, and type hints. The function should avoid mutating the input DataFrame unless explicitly specified, should operate on a configurable list of numeric features, and should return both the transformed DataFrame and a concise summary of which transformations were applied to which features.

After generating the enhanced function, review the output carefully. Test it on multiple datasets with different skew patterns and validate that the chosen transformations make sense statistically and operationally. Pay close attention to edge cases, such as columns with many zeros, negative values, or minimal variance. Save your final version for reuse in later sections and in the chapter assignment.

7.6 Missing Data



Missing data is a common challenge in data analysis and machine learning projects. If not handled correctly, it can lead to biased conclusions, inaccurate predictions, and ineffective models.

Types of Missing Data

There are three key types of missing data:

- **MCAR (Missing Completely at Random)**
 - Data is missing randomly with no pattern.
 - Least problematic; missingness is independent of the data.
- **MAR (Missing at Random)**
 - Missingness depends on other observed variables but not the missing values themselves.
 - Can often be corrected using statistical methods.
- **MNAR (Missing Not at Random)**
 - Missingness depends on unobserved values or the missing values themselves.
 - *Can be* (but not necessarily) the most problematic; it often requires domain knowledge to handle well.

It can be difficult to understand these types of missing data without examples. Next, let's cover some common reasons data might go missing.

Causes of Missing Data

Missing data can arise due to various reasons, often categorized into the following:

- **Human Error:** Data entry mistakes, forgetting to record values.
 - Type: **MCAR**
 - Examples
 - A survey respondent skips a question accidentally.
 - A data entry operator mistypes a value or leaves a field blank.
 - A researcher misplaces handwritten data before digitizing it.
- **Data Processing Issues:** Errors in merging, exporting, or loading datasets.
 - Type: **MCAR**
 - Examples
 - A CSV file is corrupted, resulting in missing values.
 - A database migration fails, causing some records to be lost.
 - A bug in an ETL (Extract, Transform, Load) pipeline causes missing entries.
- **System Constraints:** Sensors failing to capture data, software crashes.
 - Type: **MCAR**
 - Examples
 - A temperature sensor stops working intermittently.
 - A power outage interrupts data logging.
 - A website's tracking system fails to record user activity due to server downtime.
- **Respondent Unwillingness:** Participants deliberately leave fields blank based on personal preference.
 - Type: **MNAR**

- Examples
 - People avoid disclosing income in a survey.
 - Patients in a medical study refuse to answer personal questions.
 - Employees skip job satisfaction questions for fear of anonymity issues.
- **Non-Applicable Data:** Some values are not relevant to specific cases.
 - Type: **MNAR**
 - Examples
 - “Marital Status” is missing for respondents who are children.
 - “Previous Employer” is missing for someone’s first job.
 - “Car Ownership” is missing for people who do not drive.

Methods to Handle Missing Data

1. Removing Data

Method	Description	Advantages	Disadvantages
Drop Missing Rows	Remove records with missing values.	Simple and easy to implement.	Risk of losing valuable data, especially if many rows have missing values.
Drop Missing Columns	Remove entire columns with high missing percentages.	Useful when a column is mostly empty.	Could remove important features.

2. Simple Imputation (Filling Missing Data)

Fill with a Fixed Value	Replace missing values with a constant (e.g., 0, “Unknown”).	Simple and does not require computation.	May introduce bias if the value is not representative.
Fill with Mean/Median/Mode	Replace missing values with the	Works well when data is	Can distort skewed

	column's mean, median, or mode.	normally distributed.	distributions.
Forward/Backward Fill	Use the previous or next value in the column.	Useful for time-series data.	Can propagate errors if the missing values are not sequential.

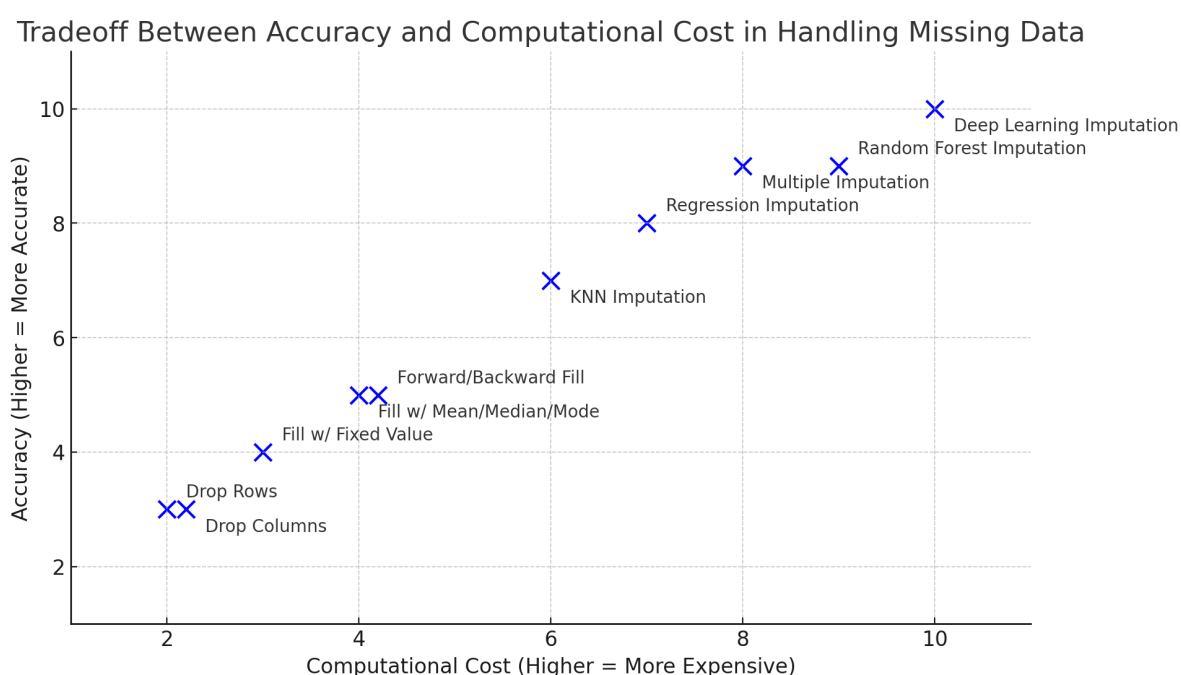
3. Advanced Imputation Techniques

K-Nearest Neighbors (KNN) Imputation	Uses the nearest neighbors to predict missing values.	Maintains relationships between variables.	Computationally expensive for large datasets.
Regression Imputation	Predicts missing values using regression models.	More accurate for MAR data.	Assumes a linear relationship that may not always hold.
Multiple Imputation	Uses probabilistic modeling to generate multiple imputed datasets and averages results.	Reduces bias and increases robustness.	Requires significant computational resources.

4. **Machine Learning Imputation:** Instead of filling missing data manually, we can train models to predict missing values.

Deep Learning-based Imputation	Uses neural networks to estimate missing values.	Can handle complex patterns in data.	Requires a large dataset and high computational power.
Random Forest Imputation	Uses multiple decision trees to predict missing values.	Handles non-linear relationships well.	Computationally expensive.

As you learn about these methods of handling missing data, you may be wondering why we do not use the most advanced and accurate techniques every time (such as deep learning-based imputation). The answer is that everything comes with a cost. The most accurate techniques are also typically the most costly in terms of computational power required. If we are building a machine learning pipeline that needs to execute within an hour, it may not be possible to use the most advanced methods. You have to make trade-off decisions, as depicted in the image below:



First Steps: Automated Missing Data Handling

The next question is how to automate this process as much as possible. Every situation is unique and needs serious thought to address the various causes of missing data correctly. However, we can automate some of it. The video below will guide you through the fairly complex function after the video:

This video can be viewed [online](#).

Let's start by pulling in the datasets that we are going to experiment with (unless you already have them from a prior section).



```
# Mount Google Drive if needed and bring in some sample data
from google.colab import drive
drive.mount('/content/drive')

import pandas as pd

# Datasets with numeric label for testing
df_insurance = pd.read_csv('/content/drive/MyDrive/Colab
Notebooks/data/insurance.csv')
df_nba = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/data/nba_salaries.csv')
df_airbnb = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/data/listings.csv')

# Dataset with categorical label for testing
df_airline = pd.read_csv('/content/drive/MyDrive/Colab
Notebooks/data/airline_satisfaction.csv')
```

Next, let's create a function that will handle dropping data so that the maximum possible amount is retained. Follow along with the video above if you don't understand how part of the function works.



```
def missing_drop(df, label="", features=[], messages=True,
row_threshold=.9, col_threshold=.5):
    import pandas as pd

    start_count = df.count().sum() # Store the initial count of non-null values

    # Drop columns with missing values beyond the specified column threshold
    df.dropna(axis=1, thresh=round(col_threshold * df.shape[0]), inplace=True)
    # Drop rows that have fewer non-null values than the row threshold allows
    df.dropna(axis=0, thresh=round(row_threshold * df.shape[1]), inplace=True)
    # If a label is specified, ensure it has no missing values
    if label != "":
        df.dropna(axis=0, subset=[label], inplace=True)

    # Function to generate a summary of missing data for each column
```

```

def generate_missing_table():
    df_results = pd.DataFrame(columns=['Missing', 'column', 'rows'])
    for feat in df:
        missing = df[feat].isna().sum() # Count missing values in column
        if missing > 0:
            memory_col = df.drop(columns=[feat]).count().sum() # Count non-null values
            if this column is dropped
                memory_rows = df.dropna(subset=[feat]).count().sum() # Count non-null values
            if rows missing this feature are dropped
                df_results.loc[feat] = [missing, memory_col, memory_rows] # Store results
    return df_results

df_results = generate_missing_table() # Generate initial missing data table

# Iteratively remove the column or rows that preserve the most non-null data
while df_results.shape[0] > 0:
    best = df_results[['column', 'rows']].max(axis=1).iloc[0] # Best preserved non-
null count on the first row (after sorting below)
    max_axis = df_results.columns[df_results.isin([best]).any()][0] # Determine
whether to drop a column or rows
    if messages: print(best, max_axis)

    df_results.sort_values(by=[max_axis], ascending=False, inplace=True) # Sort
missing data table by max_axis
    if messages: print('\n', df_results)

    # Drop the most impactful missing data (either rows or a column)
    if max_axis == 'rows':
        df.dropna(axis=0, subset=[df_results.index[0]], inplace=True) # Drop rows
missing the selected feature
    else:
        df.drop(columns=[df_results.index[0]], inplace=True) # Drop the selected
column

    df_results = generate_missing_table() # Recalculate missing data table after
dropping

# Print the percentage of non-null values retained
if messages:
    print(f'{round(df.count().sum() / start_count * 100, 2)}% ({df.count().sum()}) /
({start_count}) of non-null cells were kept.')

return df

```

So what exactly is going on in this function? Basically:

1. Initial Setup & Count
 - It calculates the total number of non-null cells (start_count).
2. Drops Columns with Too Many Missing Values
 - If a column has more than (1 - col_threshold)% missing values, it is removed.
3. Drops Rows with Too Many Missing Values
 - If a row has more than (1 - row_threshold)% missing values, it is removed.
4. Ensures the Label Column (if specified) Has No Missing Values

- If label is provided, all rows where the label column is missing are removed.
- 5. Iteratively Drops the Most Problematic Columns/Rows
 - The function creates a missing data report (`generate_missing_table`), listing how many values are missing per column.
 - While there are still missing values:
 - It identifies the option that preserves the most non-null cells after the next drop (drop a column vs. drop rows missing a feature).
 - It removes that column or those rows.
 - It updates the missing data report and repeats the process.
- 6. Final Report & Return
 - Prints the percentage of non-null data retained.
 - Returns the cleaned DataFrame.

Function Definition

Necessary Parameters

Parameter	Description
df (required)	The Pandas DataFrame that needs missing value handling.

Optional Parameters

Parameter	Default	Description
label	"" (empty string)	If provided, ensures that this label (target) column has no missing values by dropping rows where it is missing. If "", this step is skipped.
features	[] (empty list)	A list of specific columns to check for missing values. If empty, all columns are considered.
messages	True	If True, prints progress updates, including the percentage of non-null values retained. Set to False to suppress output.
row_threshold	0.9	Minimum proportion of non-missing values required to keep a row. Rows missing more than $(1 - \text{row_threshold})\%$ are removed (e.g., 0.9 → drop rows missing more than 10% of values).

Parameter	Default	Description
col_threshold	0.5	Minimum proportion of non-missing values required to keep a column. Columns missing more than (1 - col_threshold)% are removed (e.g., 0.5 → drop columns missing more than 50% of values).

Next, let's try it out on our datasets:



```
missing_drop(df_airbnb.copy()).head()

# Output:
# 263796 rows

#           Missing  column  rows
# name           24  246536  263796
# host_name      144  246656  262116
# 261780 rows

#           Missing  column  rows
# host_name      144  246344  261780
# 88.63% (261780) / (295375) of non-null cells were kept.

# See the DataFrame head() in your own notebook
```

Let's keep working through the other datasets:



```
missing_drop(df_nba.copy()).head()

# Output:
# 14505 rows

#           Missing  column  rows
# 2P%           3   14412  14784
# 3P%          12   14421  14505
# FT%          22   14431  14197
# 14414 rows

#           Missing  column  rows
# 3P%          12   14333  14414
# FT%          20   14341  14166
# 13981 column

#           Missing  column  rows
# FT%          18   13981  13856
# 93.82% (13981) / (14902) of non-null cells were kept.
```

```
# See the DataFrame head() in your own notebook
```



```
missing_drop(df_airline.copy()).head()

# Output:
# 2978201 rows

#           Missing column rows
#  Arrival Delay in Minutes    393 2857360 2978201
#  99.71% (2978201) / (2986847) of non-null cells were kept.

# See the DataFrame head() in your own notebook
```

As you've hopefully noticed from either the video or working through the code on your own, this `missing_drop` function does not drop every column or row with missing values. It only drops the biggest offenders—those where the missing percentage is over a particular threshold. For rows and columns with most of the data available, we will want to impute the remaining missing values. This is why we also have the next function below:



```
def missing_fill(df, label, features=[], row_threshold=.9, col_threshold=.5,
acceptable=0.1, mar='drop', force_impute=False, large_dataset=200000, messages=True):
    import pandas as pd, numpy as np
    from scipy import stats
    from statsmodels.stats.proportion import proportions_ztest
    pd.set_option('display.float_format', lambda x: '%.4f' % x) # Display float values
with 4 decimal places
    from IPython.display import display

    # Ensure the provided label column exists in the DataFrame
    if not label in df.columns:
        print(f'The label provided ({label}) does not exist in the DataFrame provided')
        return df

    start_count = df.count().sum() # Store the initial count of non-null values

    # Drop columns with missing data above the threshold
    df.dropna(axis=1, thresh=round(col_threshold * df.shape[0]), inplace=True)
    # Drop rows that have fewer non-null values than row_threshold allows
    df.dropna(axis=0, thresh=round(row_threshold * df.shape[1]), inplace=True)
    if label != '""': df.dropna(axis=0, subset=[label], inplace=True) # Ensure
label column has no missing values

    # If no features are specified, consider all columns as features
    if len(features) == 0: features = df.columns

    # If the label column is numeric, perform a t-test for missing vs non-missing
```

```

groups
    if pd.api.types.is_numeric_dtype(df[label]):
        df_results = pd.DataFrame(columns=['total missing', 'null x', 'non-null x', 'null
s', 'non-null s', 't', 'p'])
        for feat in features:
            missing = df[feat].isna().sum() # Count missing values
            if missing > 0:
                null = df[df[feat].isna()] # Subset where feature is missing
                nonnull = df[~df[feat].isna()] # Subset where feature is present
                t, p = stats.ttest_ind(null[label], nonnull[label]) # Perform t-test to
check for MAR vs MCAR
                df_results.loc[feat] = [round(missing), round(null[label].mean(), 6),
round(nonnull[label].mean(), 6),
round(nonnull[label].std(), 6), t, p]
            else:
                # If label is categorical, use proportions_ztest to check for MAR vs MCAR
                df_results = pd.DataFrame(columns=['total missing', 'null p', 'non-null p', 'Z',
'p'])
                for feat in features:
                    missing = df[feat].isna().sum()
                    if missing > 0:
                        null = df[df[feat].isna()]
                        nonnull = df[~df[feat].isna()]
                        for group in null[label].unique():
subset
                            p1_num = null[null[label]==group].shape[0] # Count of group in missing
subset
                            p1_den = null[null[label]!=group].shape[0] # Count of others in missing
missing subset
                            p2_num = nonnull[nonnull[label]==group].shape[0] # Count of group in non-
missing subset
                            p2_den = nonnull[nonnull[label]!=group].shape[0] # Count of others in non-

                    if p1_num < p1_den: # Avoid division by zero
                        numerators = np.array([p1_num, p2_num])
                        denominators = np.array([p1_den, p2_den])
                        z, p = proportions_ztest(numerators, denominators) # Conduct z-test
                        df_results.loc[f'{feat}_{group}'] = [round(missing), round(p1_num/p1_den,
6), round(p2_num/p2_den, 6), z, p]

                # Display the missing data analysis results
                if messages: display(df_results)

            # Determine if data is MAR (Missing at Random) or MCAR (Missing Completely at
Random)
            if df_results[df_results['p'] < 0.05].shape[0] / df_results.shape[0] >
acceptable and not force_impute:
                if mar == 'drop':
                    df.dropna(inplace=True) # Drop all rows containing missing values
                    if messages: print('null rows dropped')
                else: # Last resort: fill missing values with the median
                    for feat in df_results.index:
                        if pd.api.types.is_numeric_dtype(df[feat]):
                            df[feat].fillna(df[feat].median(), inplace=True)
                            if messages: print(f'{feat} filled with median ({df[feat].median()})')
                        else:
                            df[feat].fillna('missing', inplace=True) # Fill categorical missing values
with 'missing'
                            if messages: print(f'{feat} filled with 'missing')
                    else:
                        # If missing data is MCAR, perform imputation using either KNN or
IterativeImputer
                        from sklearn.preprocessing import OrdinalEncoder

```

```

oe = OrdinalEncoder().fit(df)
df_encoded = oe.fit_transform(df) # Convert categorical values to numeric

if df.count().sum() > large_dataset:
    from sklearn.experimental import enable_iterative_imputer
    from sklearn.impute import KNNImputer
    imp = KNNImputer() # Use K-Nearest Neighbors Imputation for large datasets
    df_imputed = imp.fit_transform(df_encoded)
    df_recoded = oe.inverse_transform(df_imputed)
    df = pd.DataFrame(df_recoded, columns=df.columns, index=df.index)
else:
    from sklearn.experimental import enable_iterative_imputer
    from sklearn.impute import IterativeImputer
    imp = IterativeImputer() # Use Iterative Imputer for smaller datasets
    df = pd.DataFrame(imp.fit_transform(df), columns=df.columns, index=df.index)

if messages: print('null values imputed')

return df

```

How does this function work?

1. Initial Setup & Dropping Rows/Columns with Too Much Missing Data
 - Removes columns that have more than $(1 - \text{col_threshold})\%$ missing values.
 - Removes rows that have more than $(1 - \text{row_threshold})\%$ missing values.
 - Ensures the label column has no missing values.
2. Statistical Testing for Missing Data Bias
 - If the label column is numeric, it performs a t-test to see whether missing values in each feature are associated with different outcomes in the label.
 - If the label column is categorical, it performs a z-test for proportions to compare missing vs. non-missing groups.
3. Determining if Missing Data is MAR (Missing at Random) or MCAR (Missing Completely at Random)
 - If too many tests show statistically significant differences ($p < 0.05$), missingness is treated as MAR.
 - If `mar='drop'`, rows with missing values are removed.
 - Otherwise, missing values are imputed using simple rules (median / "missing").
4. Imputation of Missing Data (If Needed)
 - If missingness is not treated as MAR, missing values are imputed using machine learning:
 - Large datasets ($> 200,000$ rows) → Uses KNNImputer.
 - Smaller datasets → Uses IterativeImputer.

5. Final Report & Return

- Displays statistical tests on missing data.
- Prints messages if requested.
- Returns the cleaned DataFrame.

Function Definition

Necessary Parameters

Parameter	Description
df (required)	The Pandas DataFrame that needs missing value handling.
label (required)	The name of the column to be used as the target (dependent) variable. This is important for performing statistical tests on missing data.

Optional Parameters

Parameter	Default	Description
features	[] (empty list)	List of specific columns to check for missing values. If empty, all columns are considered.
row_threshold	0.9	Minimum proportion of non-missing values required to keep a row. Rows missing more than $(1 - \text{row_threshold})\%$ are removed (e.g., 0.9 $\rightarrow$ drop rows missing more than 10% of values).
col_threshold	0.5	Minimum proportion of non-missing values required to keep a column. Columns missing more than $(1 - \text{col_threshold})\%$ are removed (e.g., 0.5 $\rightarrow$ drop columns missing more than 50% of values).
acceptable	0.1	Proportion of tests that may be statistically significant ($p < 0.05$) before missingness is treated as MAR. If more than this percentage of tests are significant, missing values are not imputed unless <code>force_impute=True</code> .

Parameter	Default	Description
mar	"drop"	If missing values are MAR, decides how to handle them: - "drop" → Remove rows with missing values. - "impute" → Fill missing values with the median (for numeric) or "missing" (for categorical).
force_impute	False	If True, forces imputation even if MAR analysis suggests dropping rows.
large_dataset	200000	Defines a large dataset threshold. If df has more than this many rows, KNNImputer is used instead of IterativeImputer for faster processing.
messages	True	If True, prints progress updates. Set to False to suppress output.

As you can tell, this is a complex function. Let's test it out:



```
df_airbnb = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/data/listings.csv')
df_airbnb = bin_categories(df_airbnb)
df_airbnb_clean = missing_fill(df_airbnb.copy(), 'average_review', mar='drop')
print(f'\nOriginal df_airbnb:\t{df_airbnb.shape}')
print(f'Cleaned df_airbnb:\t{df_airbnb_clean.shape}\n')
print(df_airbnb_clean.name.value_counts())
df_airbnb_clean.head()

# Output:
# null rows dropped

# Original df_airbnb:      (20025, 16)
# Cleaned df_airbnb:      (17452, 15)

# See the output in your own code
```

	total missing	null $\bar{x}$	non-null $\bar{x}$	null s	non-null s	t	p
name	24.0000	0.1321	0.9764	0.1436	1.4627	-2.8276	0.0047
host_name	144.0000	1.1461	0.9738	1.4722	1.4619	1.4086	0.1590



```
df_airline = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/data/airline_satisfaction.csv')
```

```
df_airline_clean = missing_fill(df_airline.copy(), 'satisfaction')
print(f'\nOriginal df_airline:\t{df_airline.shape}')
print(f'Cleaned df_airline:\t{df_airline_clean.shape}\n')
df_airline_clean.head()
```

```
# Output:
# null rows dropped

# See the output in your own code
```

	total missing	null $\hat{p}$	non-null $\hat{p}$	Z	p
Arrival Delay in Minutes_dissatisfied	393.0000	0.9171	0.8268	3.4128	0.0006



```
df_nba = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/data/nba_salaries.csv')
df_nba = basic_wrangling(df_nba)
print('...basic wrangling done\n')
df_nba_clean = missing_fill(df_nba.copy(), 'Salary', mar='impute')
print(df_nba.shape)
print(df_nba_clean.shape)
df_nba_clean.head()
```

```
# Output:
# ...basic wrangling done
#
# See the output in your own code
```

	total missing	null $\bar{x}$	non-null $\bar{x}$	null s	non-null s	t	p
3P%	12.0000	3301409.8333	8569780.2885	4995454.3839	10792808.0213	-1.6848	0.0927
2P%	3.0000	194977.0000	8487499.4341	283214.1038	10726977.0889	-1.3375	0.1817
FT%	22.0000	943020.0909	8805294.4302	1132196.2942	10838847.1753	-3.3980	0.0007

In summary, we drop only those features and rows missing too much data—much like the prior function we created. After that, we test for bias before determining the best technique for imputation. Numeric labels are tested with a t-test, while categorical labels are tested with a z-test for proportions. If there is no evidence of bias, then we can impute missing values using `KNNImputer` or `IterativeImputer`. If there is evidence of bias, then we can choose whether to drop the rows entirely or fill missing values using a simpler approach (such as the median for numeric features).

Before using AI tools, it is important to understand the core logic of missing data handling by working through a human-designed solution first. In this section, you implemented two functions that represent a practical baseline approach:

- **missing_drop()** removes the biggest offenders (rows/columns missing too much data) while trying to retain as many non-null cells as possible.
- **missing_fill()** goes further by testing for missing-data bias and then choosing whether to drop or impute remaining missing values using a mix of statistical tests and machine-learning imputers.

These functions are intentionally “first steps.” They force you to practice the essentials: identifying missingness, setting thresholds, thinking about trade-offs (accuracy vs. cost), and understanding why imputation is not always appropriate. Once you can follow and explain this baseline approach, you are ready to responsibly use AI to improve it.

AI Assistance: Expanding and Refactoring Our Missing-Data Functions

Now that you understand the baseline, we can use AI to improve the structure, efficiency, safety, and capabilities of our missing-data automation. The goal is not to replace your thinking, but to accelerate iteration and help you implement best practices that are easy to miss when you are learning.

In particular, an AI assistant can help you redesign *missing_drop()* and *missing_fill()* so they are more reusable, easier to maintain, and safer for real-world use—while preserving the original behavior.

AI Prompt:

You are an expert Python data engineer and machine learning practitioner. I have two existing Pandas-based functions for handling missing data: (1) `missing_drop()`, which drops rows/columns with excessive missingness while trying to retain the maximum number of non-null cells, and (2) `missing_fill()`, which tests for missing-data bias using statistical tests (t-test for numeric targets, proportion z-tests for categorical targets) and then decides whether to drop or impute remaining missing values (median/"missing" for MAR, or ML-based imputation for MCAR using KNNImputer for large datasets and IterativeImputer for smaller datasets).

Redesign and expand these functions into a robust, efficient, and reusable missing-data module while preserving the original high-level behavior. Improve code quality, correctness, efficiency, and extensibility. Requirements:

- 1) Do not mutate the input DataFrame unless explicitly specified (add an `inplace` parameter or return a new DataFrame by default).
- 2) Add clear docstrings, type hints, and consistent parameter naming. Validate inputs and handle edge cases safely (empty DataFrames, missing label column, all-null columns, non-unique indices, mixed dtypes, columns listed in features not found, etc.).
- 3) Replace print statements with configurable logging (Python logging module). Allow log level control and an option to silence logs.
- 4) Support selecting a subset of columns to process (features) without silently ignoring it. Ensure the functions work correctly when features is empty (default all columns) and when a label column is provided.
- 5) Improve `missing_drop()`: fix any logical bugs; ensure the iterative step correctly evaluates whether dropping a column or dropping rows for a specific column retains more non-null cells; avoid expensive recomputation where

possible; and ensure it terminates reliably. Provide a concise summary report (e.g., starting shape, ending shape, % non-null cells retained, and what was dropped).

6) Improve `missing_fill()`: avoid data leakage by supporting a train/test workflow (fit imputers on training data only, apply to test); add reproducibility controls (`random_state`); handle categorical encoding robustly (avoid fitting `OrdinalEncoder` twice; avoid encoding label if inappropriate; preserve column dtypes where reasonable); and make the MCAR/MAR decision logic clearer and correct. If you keep the statistical testing approach, ensure it is implemented correctly and explain assumptions. If you change the decision rule, justify it.

7) Add optional strategies: per-column imputation rules (mean/median/mode/constant), group-wise imputation, datetime-safe handling, and the ability to exclude columns (like IDs) from imputation.

8) Use vectorized Pandas operations where possible and avoid repeated full scans of the `DataFrame`. Keep computational cost in mind for large datasets.

9) Include inline comments explaining major design decisions. Provide example usage showing: (a) drop-only workflow, (b) drop+impute workflow, and (c) train/test safe workflow.

Here is my current code to refactor and expand (keep the same overall intent, but you may reorganize and rename as needed):

[PASTE `missing_drop()` AND `missing_fill()` HERE]

7.7 Managing Outliers



Outliers are extreme values that deviate significantly from the rest of the dataset. They can skew results, impact model performance, and distort statistical analyses. Properly identifying and handling outliers is crucial for effective data analysis.

Identifying Outliers

There are three primary techniques for identifying outliers, which we discuss below:

1. **Empirical Rule (Z-Score Method)**
2. **Tukey's Boxplot Rule (IQR Method)**
3. **DBSCAN Clustering Algorithm**

Each method has its own strengths and weaknesses, making it suitable for different types of datasets.

Empirical Rule (a.k.a. Z-Score Method; a.k.a., 68-95-99.7 Rule)

The Empirical Rule is appropriate when normality assumptions are reasonable. It states that for a normal distribution:

- **68%** of values fall within 1 standard deviation (σ) of the mean.
- **95%** of values fall within 2 standard deviations (σ) of the mean.
- **99.7%** of values fall within 3 standard deviations (σ) of the mean.

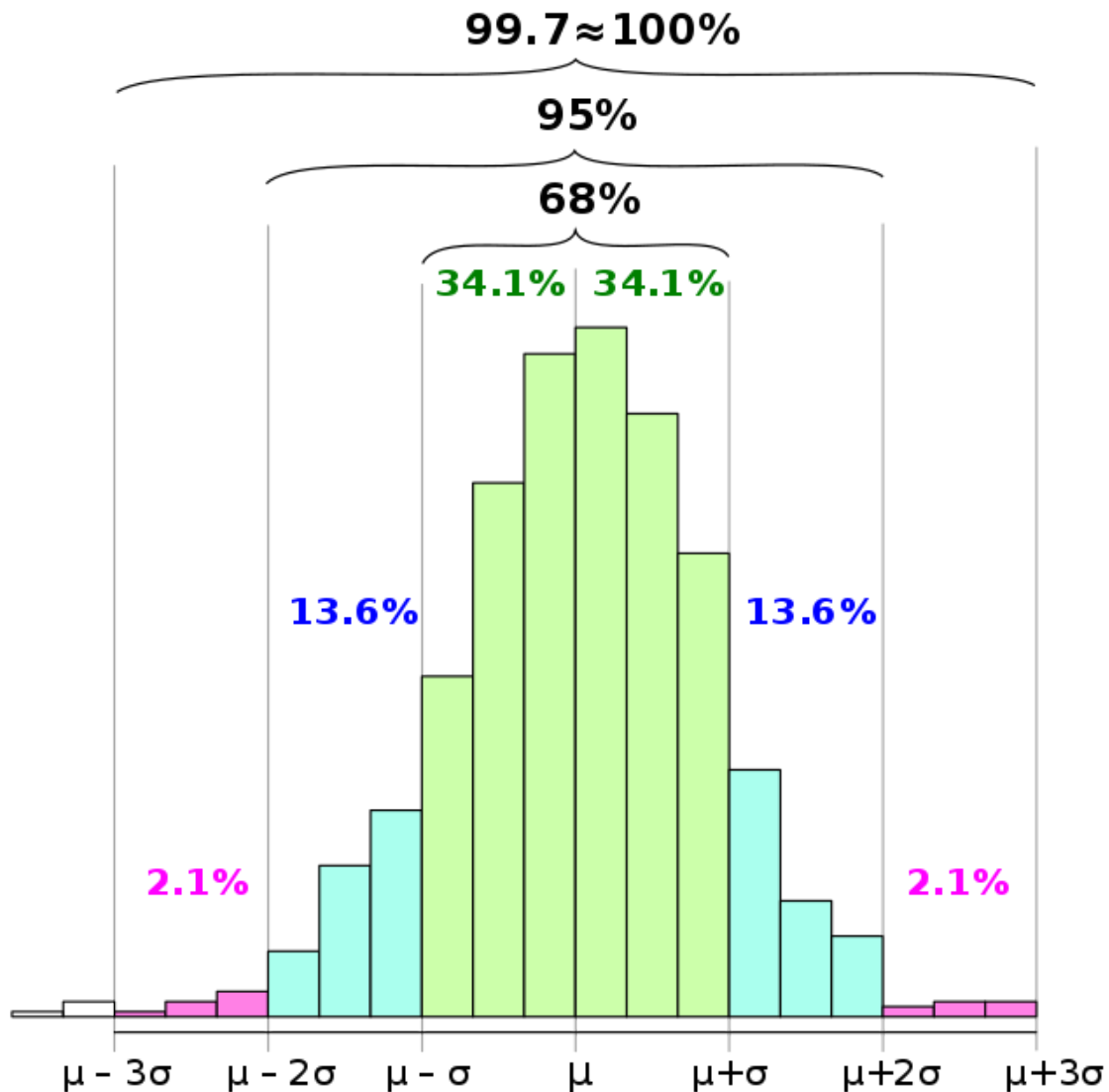


Figure 7.2: Normal Distribution and Outliers

This technique is effective when the data follow a normal distribution, and it is easy to compute and interpret. However, it does not reliably identify outliers when the data are skewed or non-normal. In addition, extreme outliers can distort the mean and standard deviation, which can reduce the method's accuracy.

Tukey's Boxplot Rule (a.k.a., Interquartile Range - IQR)

The Tukey box plot (or adjusted box plot) A particular case of a box plot intended for skewed distributions where the max/min (or “whiskers”) of the plot are defined as the lowest/highest data point that is still within $1.5 \times \text{interquartile range}$ ($Q_3 - Q_1$). is intended for skewed distributions where the max/min (or *whiskers*) are defined as the lowest/highest data points that are still within $1.5 \times$ the interquartile range (IQR, defined as $Q_3 - Q_1$), as depicted in the image below:

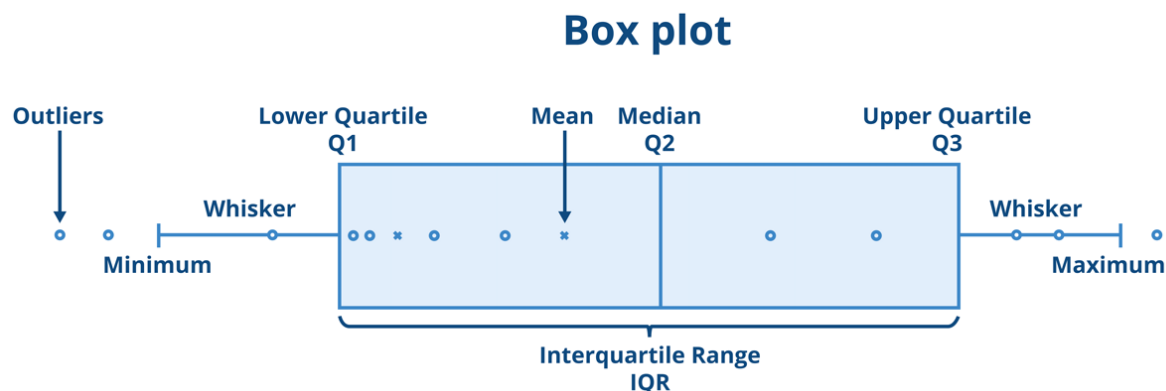


Figure 7.3: Box Plot Details Mapped to Normal Distribution

Conveniently, the box plot function in the Matplotlib package (`plt.boxplot()`) defaults to Tukey's box plot rule with a $1.5 \times \text{IQR}$ distance for the minimum and maximum values. We can generate the box plot without displaying it so that we can retrieve the $1.5 \times \text{IQR}$ minimum and maximum values. Then, as

before, we can use those values to generate replacement values at the theoretical minimum and maximum.

Boxplot Returned Values

The `boxplot()` function returns a dictionary containing the following information:

- boxes: the main body of the box plot showing the quartiles and the median's confidence intervals (if enabled).
- medians: the horizontal lines at the median of each box.
- whiskers: the vertical lines extending to the most extreme, non-outlier data points.
- caps: the horizontal lines at the ends of the whiskers.
- fliers: the points representing data that extend beyond the whiskers.
- means: the points or lines representing the means.

In summary, the IQR (Tukey's box plot) method works well when the data are skewed or non-normal, and it is not as affected by extreme outliers. However, it is less effective when the data have multiple peaks (a.k.a. "multimodal" distributions). It also assumes a roughly symmetric distribution around the median, which may not always be true.

DBSCAN Clustering Algorithm

Another limitation of both the Empirical Rule and Tukey's box plot methods is that they identify outliers one feature at a time. A value may appear to be an outlier within a single feature, but when combined with other values in the record, it may be reasonable. For example, imagine an employee dataset where one person's salary is higher than all others. If you only examine salary, that person may appear to be an outlier. But if you consider other variables,

you might realize the person is the CEO and has worked at the company longer than everyone else. In that context, you might decide the salary is not problematic (or you might still treat it as an outlier, depending on your goals).

This is the primary limitation addressed by clustering-based outlier detection techniques such as DBSCAN (Density-Based Spatial Clustering of Applications with Noise). DBSCAN is an unsupervised learning algorithm that clusters data points based on density. Outliers are points that are not assigned to any cluster.

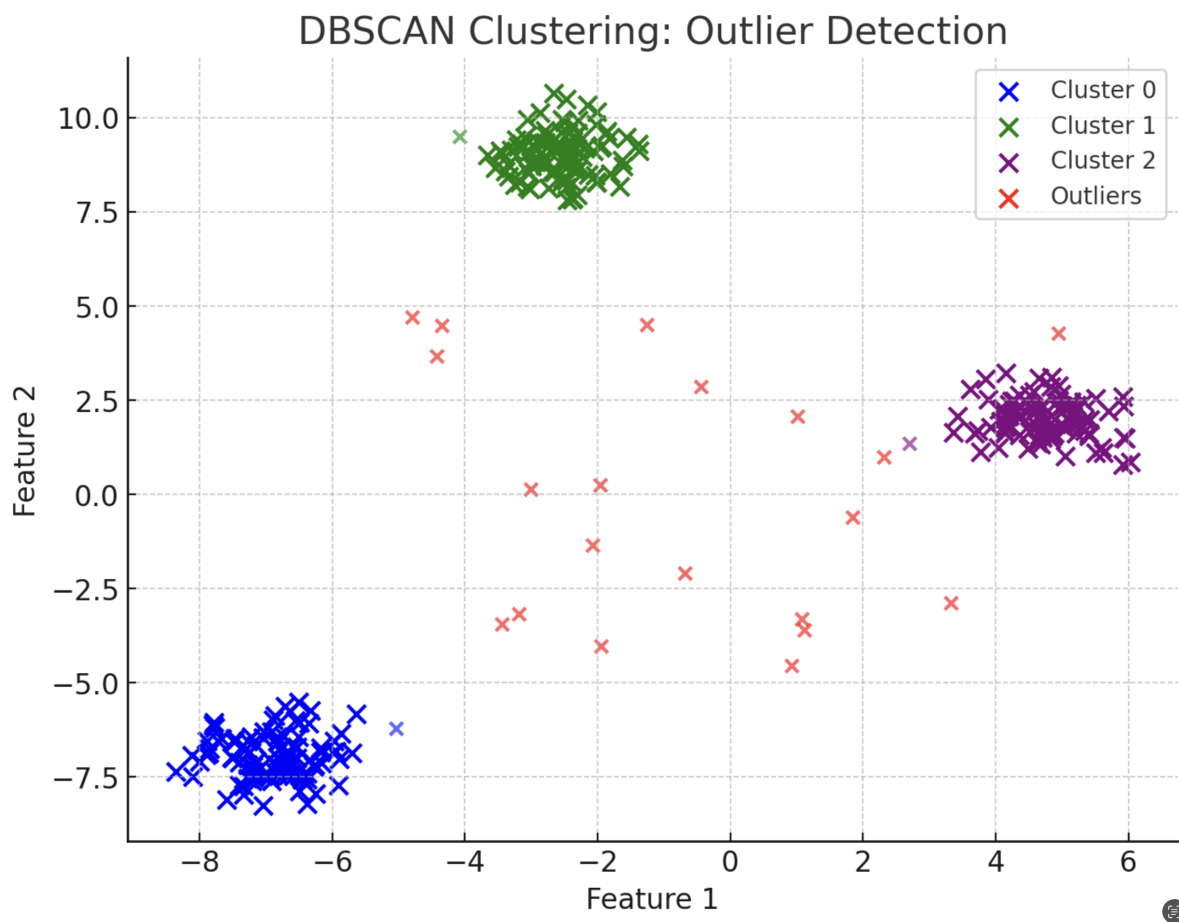


Figure 7.4: DBSCAN Clustering and Outliers

Addressing Outliers

Once outliers have been identified, the next step is to decide how to handle them. The best approach depends on the context of the data, the impact of outliers, and the goals of the analysis. Let's briefly review the options.

Remove Outliers

The first option is to remove outlier records altogether.

```
# Remove outliers detected by Z-Score
df_cleaned = df[df['Z-Score'].abs() <= 3]

# Remove outliers detected by IQR
df_cleaned = df[(df['Value'] >= lower_bound) & (df['Value'] <=
upper_bound)]
```

This method is clean and simple, but it may eliminate useful data that you would benefit from retaining. Only do this if you are confident those values will not appear in future data, or if you know they are not relevant to your analysis.

Cap or Winsorize Outliers (Capping)

Next, you can cap outliers by replacing extreme values with a theoretical minimum or maximum that represents the meaningful edge of the range.

```
# Cap outliers using percentile-based Winsorization
lower_cap = df['Value'].quantile(0.05)
upper_cap = df['Value'].quantile(0.95)

df['Capped'] = np.where(df['Value'] < lower_cap, lower_cap,
                        np.where(df['Value'] > upper_cap, upper_cap, df['Value']))
```

The advantage of this method is that it keeps as much data as possible. The disadvantage is that it can create artificial “bumps” at the edges of the distribution, making the data appear multimodal.

Transform Data

We covered earlier how you can mathematically transform a skewed feature so that it is closer to normal. Doing this often reduces the influence of outliers. Although you have seen this before, here is a brief example again:

```
# Log Transformation
df['Log_Transformed'] = np.log1p(df['Value']) # log1p avoids log(0) issues

# Square Root Transformation
df['Sqrt_Transformed'] = np.sqrt(df['Value'])

# Box-Cox Transformation (only for positive values)
from scipy.stats import boxcox
df['BoxCox_Transformed'], lambda_ = boxcox(df['Value'] + 1) # Avoid zero values
```

This approach is ideal for skewed features, but it may not be effective enough when the skewness is extreme.

Use Robust Models

Another option is to keep the outliers and plan to use algorithms later in the modeling phase that are less sensitive to extreme values. You might wonder why we do not always use this approach. In some cases, linear algorithms that benefit from roughly normal data can outperform robust methods (such as decision trees) when outliers are addressed appropriately.

Automating in Functions

This video can be viewed [online](#).

Now that we have an idea of how to identify and address outliers, let's create a couple of functions to put this together. In particular, we will create two functions: one to handle the one-at-a-time techniques (Empirical Rule and

Tukey's box plot) and one to handle the more advanced all-at-once technique (DBSCAN).

One-At-A-Time

Let's begin with a function that uses either the Empirical Rule or Tukey's box plot to identify and clean outliers, depending on whether the skewness of each numeric feature is within acceptable boundaries (-1 to 1).



```
# Documentation
# required:
#   Pandas DataFrame
#   A list of features to clean outliers
# optional:
#   The method of cleaning:
#       remove = delete outlier rows
#       replace = replace the value with the theoretical min/max
#       impute = fill in a predicted amount based on a linear model
#       null = keep the rows but replace the outliers with null
#   Whether to include skip messages

def clean_outlier(df, features=[], method="remove", messages=True,
skew_threshold=1):
    import pandas as pd, numpy as np

    for feat in features:
        if feat in df.columns:
            if pd.api.types.is_numeric_dtype(df[feat]):
                if df[feat].nunique() != 1:
                    if not all(df[feat].value_counts().index.isin([0, 1])):
                        skew = df[feat].skew()
                        if skew < (-1 * skew_threshold) or skew > skew_threshold: # Tukey
boxplot rule: < 1.5 * IQR < is an outlier
                            q1 = df[feat].quantile(0.25)
                            q3 = df[feat].quantile(0.75)
                            min = q1 - (1.5 * (q3 - q1))
                            max = q3 + (1.5 * (q3 - q1))
                        else: # Empirical rule: any value > 3 std from the mean (or < 3)
is an outlier
                            min = df[feat].mean() - (df[feat].std() * 3)
                            max = df[feat].mean() + (df[feat].std() * 3)

                            min_count = df.loc[df[feat] < min].shape[0]
                            max_count = df.loc[df[feat] > max].shape[0]
                            if messages: print(f'{feat} has {max_count} values above max={max} and
{min_count} below min={min}')

    if min_count > 0 or max_count > 0:
        if method == "remove": # Remove the rows with outliers
            df = df[df[feat] > min]
            df = df[df[feat] < max]
```

```

elif method == "replace": # Replace the outliers with the
min/max cutoff
    df.loc[df[feat] < min, feat] = min
    df.loc[df[feat] > max, feat] = max
elif method == "impute": # Impute the outliers by deleting
them and then predicting the values based on a linear regression
    df.loc[df[feat] < min, feat] = np.nan
    df.loc[df[feat] > max, feat] = np.nan

    from sklearn.experimental import enable_iterative_imputer
    from sklearn.impute import IterativeImputer
    imp = IterativeImputer(max_iter=10)
    df_temp = df.copy()
    df_temp = bin_categories(df_temp, features=df_temp.columns,
messages=False)
    df_temp = basic_wrangling(df_temp, features=df_temp.columns,
messages=False)
    df_temp = pd.get_dummies(df_temp, drop_first=True)
    df_temp = pd.DataFrame(imp.fit_transform(df_temp),
columns=df_temp.columns, index=df_temp.index, dtype='float')
    df_temp.columns = df_temp.columns.get_level_values(0)
    df_temp.index = df_temp.index.astype('int64')

    # Save only the column from df_temp that we are iterating on in the
main loop because we may not want every new column
    df[feat] = df_temp[feat]
elif method == "null":
    df.loc[df[feat] < min, feat] = np.nan
    df.loc[df[feat] > max, feat] = np.nan
else:
    if messages: print(f'{feat} is a dummy code (0/1) and was ignored')
else:
    if messages: print(f'{feat} has only one value ({df[feat].unique()[0]}) and
was ignored')
else:
    if messages: print(f'{feat} is categorical and was ignored')
else:
    if messages: print(f'{feat} is not found in the DataFrame provided')

return df

```

Let's review what is happening in this function. If the list below is not detailed enough, I recommend following along with the video above to understand it more completely.

1. Iterates Through the Given Features

- Checks whether each feature exists in the DataFrame.
- Skips features that are:
 - Categorical (non-numeric data).
 - Binary (0/1) (likely dummy variables).
 - Constant (only one unique value).

2. Detects Outliers Using Two Methods

- Tukey's Boxplot Rule (for Skewed Data)
 - If the data are highly skewed ($\text{skew} > \text{skew_threshold}$), outliers are defined as values beyond 1.5 times the IQR (Interquartile Range).
 - $\text{min} = Q1 - (1.5 * \text{IQR})$
 - $\text{max} = Q3 + (1.5 * \text{IQR})$
 - Empirical Rule (for Normally Distributed Data)
 - If the data are not highly skewed, outliers are values more than 3 standard deviations from the mean.
 - $\text{min} = \text{mean} - (3 * \text{std})$
 - $\text{max} = \text{mean} + (3 * \text{std})$
3. Counts and Logs the Number of Outliers
- Calculates how many values fall below min or above max.
 - If messages=True, prints how many outliers exist in each column.
4. Handles Outliers Based on the method Parameter
- "remove": drops rows with outliers.
 - "replace": replaces outliers with the min/max cutoff.
 - "impute": uses IterativeImputer to predict missing values.
 - "null": replaces outliers with NaN.
5. Returns the Cleaned DataFrame
- The function modifies and returns the DataFrame with outliers handled based on the selected method.

Let's try our new function on the datasets we have imported. First, let's examine the properties of the insurance dataset and the BMI feature in particular to see how it changes:



```
df_insurance = pd.read_csv('/content/drive/MyDrive/Colab
Notebooks/data/insurance.csv')
print(df_insurance.shape)
df_top5 = df_insurance.sort_values(by=['bmi'], ascending=False)['bmi'].head()
```

```
print(df_top5)

# Output:
# (1338, 7)
# 1317    53.1300
# 1047    52.5800
# 847     50.3800
# 116     49.0600
# 286     48.0700
# Name: bmi, dtype: float64
```

Keep these numbers for reference, and now let's run the function on this dataset:



```
df_insurance = pd.read_csv('/content/drive/MyDrive/Colab
Notebooks/data/insurance.csv')
df_insurance = clean_outlier(df_insurance, features=df_insurance.columns,
method='remove')
print(df_insurance.shape)
df_insurance.loc[df_insurance.index.isin(df_top5.index)].head()

# Output:
# age has 0 values above max=81.3569065487098 and 0 below min=-2.9428557265872257
# sex is categorical and was ignored
# bmi has 4 values above max=48.957957596023604 and 0 below min=12.368836125949496
# children has 18 values above max=4.716345622635248 and 0 below
min=-2.5229423242844238
# smoker is categorical and was ignored
# region is categorical and was ignored
# charges has 129 values above max=35232.529987500006 and 0 below
min=-13588.807712500002
# (1187, 7)
#    age    sex    bmi  children smoker    region    charges
# 286   46  female  48.0700         2     no  northeast  9432.9253
```

After running the insurance dataset through our new function, we can see that bmi, children, and charges each have some outliers. Because children is an integer with only a few possible values, you may prefer to remove it from outlier detection. Notice that after removing the four outliers from BMI, only one of the original top five BMI records remains. After removing the outliers for charges and children, the dataset dropped from 1,338 records to 1,187.

Let's run this function again using the "replace" technique instead of "remove". Let's also ignore the children column:



```

df_insurance = pd.read_csv('/content/drive/MyDrive/Colab
Notebooks/data/insurance.csv')
df_insurance = clean_outlier(df_insurance, features=df_insurance.drop(columns=
['children']).columns, method='replace')
print(df_insurance.shape)
print(df_insurance.loc[df_insurance.index.isin(df_top5.index)].head())

# Output:
# age has 0 values above max=81.3569065487098 and 0 below min=-2.9428557265872257
# sex is categorical and was ignored
# bmi has 4 values above max=48.957957596023604 and 0 below min=12.368836125949496
# smoker is categorical and was ignored
# region is categorical and was ignored
# charges has 139 values above max=34489.350562499996 and 0 below min=-13109.1508975
# (1338, 7)
#      age      sex      bmi  children  smoker      region      charges
# 116    58   male  48.9580         0      no  southeast  11381.3254
# 286    46  female  48.0700         2      no  northeast   9432.9253
# 847    23   male  48.9580         1      no  southeast   2438.0552
# 1047   22   male  48.9580         1     yes  southeast  34489.3506
# 1317   18   male  48.9580         0      no  southeast   1163.4627

```

Notice that this time we kept all 1,338 records, but the values that were previously dropped for bmi and charges have now been replaced with the theoretical maximum or minimum. You can see a potential downside of this practice if you have too many outliers. Take a look at the histplots() of charges before and after replacing outliers in the output image of the code below:



```

import seaborn as sns
import matplotlib.pyplot as plt

# Load dataset
df_insurance = pd.read_csv('/content/drive/MyDrive/Colab
Notebooks/data/insurance.csv')

# Save original 'bmi' values before outlier cleaning
df_original = df_insurance.copy()

# Apply outlier cleaning (replacing outliers with min/max cutoff)
df_insurance = clean_outlier(df_insurance, features=df_insurance.drop(columns=
['children']).columns, method='replace', messages=False)

# Create subplots: 1 row, 2 columns
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Plot histogram for original 'bmi' values
sns.histplot(df_original['charges'], ax=axes[0], kde=True)
axes[0].set_title('"Charges Distribution (Before Cleaning)"')
axes[0].set_xlabel('"Charges"')

```

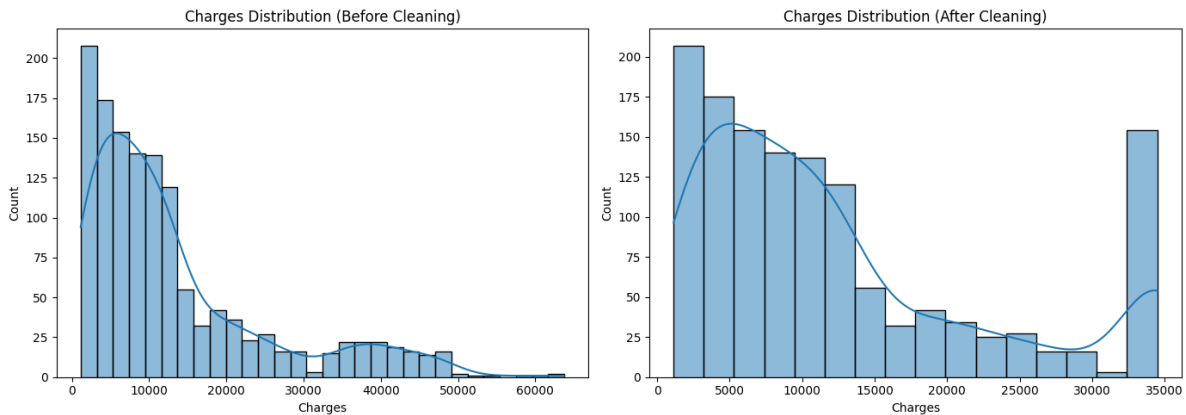
```

axes[0].set_ylabel(""Count"");

# Plot histogram for cleaned 'bmi' values
sns.histplot(df_insurance['charges'], ax=axes[1], kde=True)
axes[1].set_title(""Charges Distribution (After Cleaning)"")
axes[1].set_xlabel(""Charges"");
axes[1].set_ylabel(""Count"");

# Adjust layout for better visibility
plt.tight_layout()
plt.show()

```



Finally, let's try it again using imputation:

```

df_insurance = pd.read_csv('/content/drive/MyDrive/Colab
Notebooks/data/insurance.csv')
df_insurance = clean_outlier(df_insurance, features=df_insurance.columns,
method='impute')
print(df_insurance.loc[df_insurance.index.isin(df_top5.index)].head())

# Output
# age has 0 values above max=81.3569065487098 and 0 below min=-2.9428557265872257
# sex is categorical and was ignored
# bmi has 4 values above max=48.957957596023604 and 0 below min=12.368836125949496
# children has 18 values above max=4.711396007088629 and 0 below
min=-2.5215604316028286
# smoker is categorical and was ignored
# region is categorical and was ignored
# charges has 139 values above max=34489.350562499996 and 0 below min=-13109.1508975
#
#   age  sex  bmi  children  smoker  region  charges
# 116  58  male  33.6658    0.0000    no  southeast  11381.3254
# 286  46 female  48.0700    2.0000    no  northeast  9432.9253
# 847  23  male  31.9898    1.0000    no  southeast  2438.0552
# 1047 22  male  36.8380    1.0000   yes  southeast  18595.6066
# 1317 18  male  31.8094    0.0000    no  southeast  1163.4627

```

All-At-Once

This video can be viewed [online](#).

Next, let's create another function that uses the DBSCAN clustering algorithm.



```
# Documentation
# required: a Pandas DataFrame
# optional:
# messages = True          -&gt; whether or not to include detailed information about
outliers
# drop_percent = 0.02      -&gt; the percent of the dataset you want dropped as
outliers. The eps parameter will be automatically adjusted to accomplish this
# distance = 'euclidean' -&gt; the distance metric used for the clustering
algorithm. Options: ['cityblock', 'cosine', 'euclidean', 'l1', 'l2', 'manhattan']

def clean_outliers(df, messages=True, drop_percent=0.02, distance='manhattan',
min_samples=5):
    import pandas as pd, numpy as np, seaborn as sns, matplotlib.pyplot as plt
    from sklearn.cluster import DBSCAN
    from sklearn import preprocessing

    # Clean the dataset first
    if messages: print(f""{df.shape[1]} - df.dropna(axis='columns').shape[1]}
columns were dropped first due to missing data"")
    df.dropna(axis='columns', inplace=True)
    if messages: print(f""{df.shape[0]} - df.dropna().shape[0]} rows were dropped
first due to missing data"")
    df.dropna(inplace=True)
    df_temp = df.copy()
    df_temp = bin_categories(df_temp, features=df_temp.columns, messages=False)
    df_temp = basic_wrangling(df_temp, features=df_temp.columns, messages=False)
    df_temp = pd.get_dummies(df_temp, drop_first=True)
    # Normalize the dataset
    df_temp = pd.DataFrame(preprocessing.MinMaxScaler().fit_transform(df_temp),
columns=df_temp.columns, index=df_temp.index)

    # Calculate the number of outliers based on a range of eps values
    outliers_per_eps = []
    outliers = df_temp.shape[0]
    eps = 0

    if df_temp.shape[0] < 500:
        iterator = 0.01
    elif df_temp.shape[0] < 2000:
        iterator = 0.05
    elif df_temp.shape[0] < 10000:
        iterator = 0.1
    elif df_temp.shape[0] < 25000:
        iterator = 0.2

    while outliers > 0:
        eps += iterator
```



```

        db = DBSCAN(metric=distance, min_samples=min_samples, eps=eps).fit(df_temp)
        outliers = np.count_nonzero(db.labels_ == -1)
        outliers_per_eps.append(outliers)
        if messages: print(f'eps: {round(eps, 2)}, outliers: {outliers}, percent:
{round((outliers / df_temp.shape[0])*100, 3)}%')

        drops = min(outliers_per_eps, key=lambda x:abs(x-round(df_temp.shape[0] *
drop_percent)))
        eps = (outliers_per_eps.index(drops) + 1) * iterator
        db = DBSCAN(metric=distance, min_samples=min_samples, eps=eps).fit(df_temp)
        df['outlier'] = db.labels_

    if messages:
        print(f"&quot;{df[df['outlier'] == -1].shape[0]} outlier rows removed from the
DataFrame&quot;")
        sns.lineplot(x=range(1, len(outliers_per_eps) + 1), y=outliers_per_eps)
        sns.scatterplot(x=[eps/iterator], y=[drops])
        plt.xlabel(f'eps (divide by {iterator})')
        plt.ylabel('Number of Outliers')
        plt.show()

    # Drop rows that are outliers
    df = df[df['outlier'] != -1]
    return df

```

This function is complicated, but it is well worth the time it takes to understand it. Follow along with the video above if needed. To summarize, this function does the following:

1. Data Preprocessing

- Removes columns with missing data.
- Removes rows with missing data.
- Encodes categorical variables:
 - Uses `bin_categories()` to group rare categories.
 - Uses `basic_wrangling()` to remove redundant or low-value columns.
 - Converts categorical features into dummy variables (`pd.get_dummies()`).
- Normalizes all features using Min-Max Scaling to ensure numerical stability.

2. Finding the Optimal eps (Epsilon) for DBSCAN

- Initial Setup: Defines an iterator step for eps:
 - Small datasets (<500 rows): eps increases in 0.01 steps.

- Medium datasets (500–2,000 rows): eps increases in 0.05 steps.
- Large datasets (2,000–10,000 rows): eps increases in 0.1 steps.
- Very large datasets (10,000–25,000 rows): eps increases in 0.2 steps.
- Iterates through eps values:
 - Runs DBSCAN with increasing eps.
 - Counts outliers (db.labels_ == -1).
 - Logs the number of detected outliers at each step.

3. Selecting the Best eps

- Finds the eps value that results in approximately drop_percent% of the dataset being labeled as outliers.
- Re-runs DBSCAN with the chosen eps.

4. Removing Outliers

- Assigns an “outlier” label (-1) to detected outliers in df['outlier'].
- Drops rows where outlier == -1.

5. Visualizing Outlier Detection (Optional)

- Plots a line graph showing the number of outliers detected at each eps value.
- Marks the chosen eps on the graph.

6. Optional Parameters

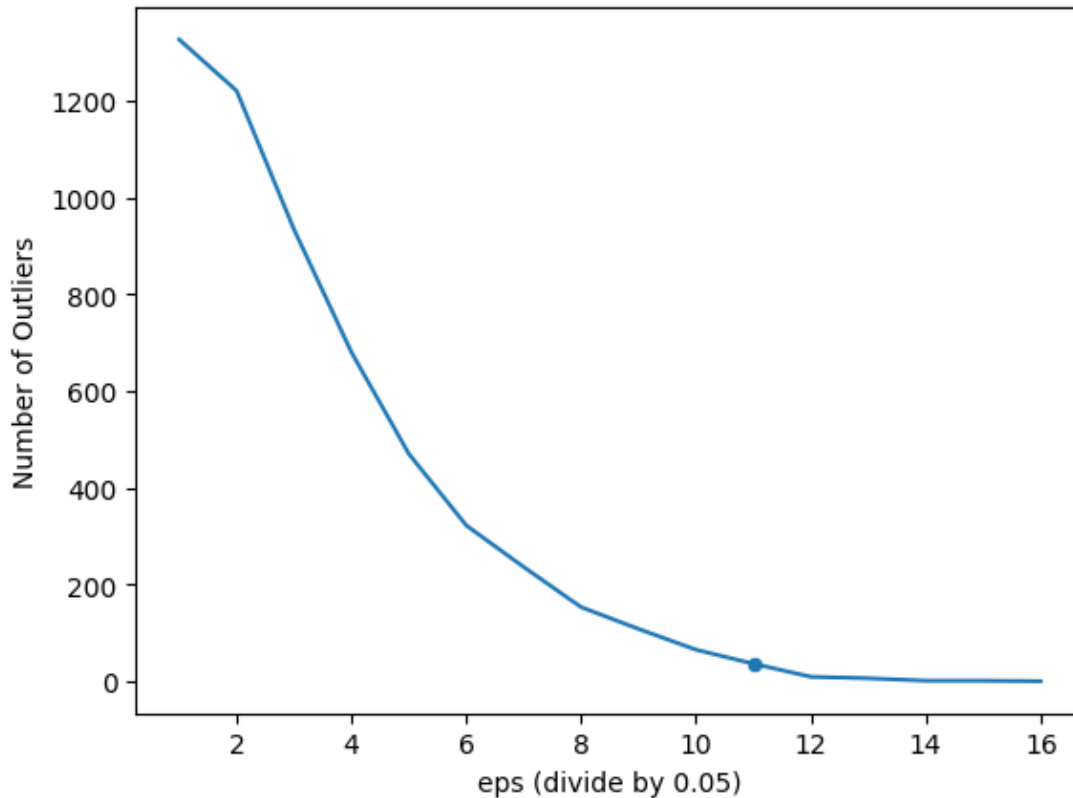
- messages=True: If True, prints detailed logs and shows plots of the outlier detection process.
- drop_percent=0.02: The percentage of the dataset to be removed as outliers (e.g., 0.02 → removes ~2% of rows as outliers).
- distance=manhattan: The distance metric used for DBSCAN clustering. Options: ['cityblock', 'cosine', 'euclidean', 'l1', 'l2', 'manhattan'].
- min_samples=5: The minimum number of points required to form a cluster. Lower values detect more outliers; higher values detect

fewer outliers.

Let's go ahead and use this function on the insurance dataset:

```
df_insurance = pd.read_csv('/content/drive/MyDrive/Colab
Notebooks/data/insurance.csv')
df_insurance = clean_outliers(df_insurance, min_samples=5)
print(df_insurance.shape)
print(df_insurance.head())

# Output
# 0 columns were dropped first due to missing data
# 0 rows were dropped first due to missing data
# eps: 0.05, outliers: 1327, percent: 99.178%
# eps: 0.1, outliers: 1221, percent: 91.256%
# eps: 0.15, outliers: 934, percent: 69.806%
# eps: 0.2, outliers: 680, percent: 50.822%
# eps: 0.25, outliers: 470, percent: 35.127%
# eps: 0.3, outliers: 322, percent: 24.066%
# eps: 0.35, outliers: 236, percent: 17.638%
# eps: 0.4, outliers: 153, percent: 11.435%
# eps: 0.45, outliers: 108, percent: 8.072%
# eps: 0.5, outliers: 65, percent: 4.858%
# eps: 0.55, outliers: 36, percent: 2.691%
# eps: 0.6, outliers: 9, percent: 0.673%
# eps: 0.65, outliers: 6, percent: 0.448%
# eps: 0.7, outliers: 1, percent: 0.075%
# eps: 0.75, outliers: 1, percent: 0.075%
# eps: 0.8, outliers: 0, percent: 0.0%
# 36 outlier rows removed from the DataFrame
#
# (1302, 8)
#   age  sex    bmi  children  smoker  region    charges  outlier
# 0   19 female  27.9000         0    yes southwest  16884.9240        0
# 1   18  male  33.7700         1    no  southeast   1725.5523        1
# 2   28  male  33.0000         3    no  southeast   4449.4620        1
# 3   33  male  22.7050         0    no northwest  21984.4706        2
# 4   32  male  28.8800         0    no northwest   3866.8552        2
```



As you can see, using the default drop_percent value of 0.02 drops the 36 (1,338 - 1,302) most extreme records. This is a better way to remove outliers if you are not planning to use a linear algorithm later in the modeling phase.

AI Assistance: Expanding and Optimizing Outlier Handling

In this section, you created functions that automate outlier detection and cleaning. These are the **first steps** you should be able to perform on your own so you understand the fundamentals of identifying and managing extreme values.

However, real-world datasets often require more flexibility than a single function can provide. For example, you may need to:

- Choose different detection methods based on feature type (continuous numeric vs. count data vs. ordinal vs. binary).
- Handle grouped or segmented outliers (e.g., detect BMI outliers separately by gender or age group).
- Detect multivariate outliers more broadly than DBSCAN (e.g., Isolation Forest, Local Outlier Factor, Robust Covariance).
- Automate method selection based on distributional diagnostics (skewness, kurtosis, heavy tails, multimodality, zero inflation).
- Improve performance and reliability for large datasets (vectorization, minimized copying, safer handling of missing data, and clearer reporting).

AI can help you refactor and expand your existing functions into a more complete outlier-handling toolkit. The goal is not to replace your understanding, but to accelerate iteration and add capabilities that would otherwise take substantial time to engineer manually.

AI Prompt:

You are helping me improve and expand my Python >outlier detection and handling. I will provide two existing functions: (1) `clean_outlier()` for one-feature-at-a-time outlier handling using either the Empirical Rule (Z-score style) or Tukey's IQR rule, and (2) `clean_outliers()` for all-features-at-once detection using DBSCAN. Your task is to refactor, optimize, and expand these functions into production-quality utilities while preserving their original purpose and making them easier to teach.

Requirements: (1) Keep the API beginner-friendly, but allow advanced options. (2) Make the code more efficient (vectorized where possible, avoid repeated expensive operations, avoid unnecessary copies). (3) Add robust input validation and clear error messages. (4) Improve readability and structure (docstrings, consistent naming, avoid shadowing built-ins like

min/max, avoid ambiguous variables). (5) Ensure missing data handling is safe (avoid silently dropping too much without reporting). (6) Provide consistent, optional logging (a messages flag that prints a concise summary).

Enhancements to add: Add support for multiple univariate detection methods (Z-score, modified Z-score using MAD, IQR/Tukey, percentile/quantile caps), automatic method selection based on distribution diagnostics (skewness/kurtosis/zero-inflation), and separate handling for continuous variables vs. count variables (e.g., children) vs. binary variables. Add group-wise outlier detection (e.g., detect within groups using a groupby key list). Add optional multivariate outlier methods beyond DBSCAN (IsolationForest and LocalOutlierFactor) with a consistent interface.

Outlier actions: Keep and improve the existing actions (remove, replace/cap, null, impute). For imputation, provide at least two options: (a) simple imputation (median/mean by feature or group) and (b) model-based imputation (IterativeImputer) with careful preprocessing (encoding categoricals safely). Ensure the user can choose whether to treat outliers as missing before imputing.

Outputs: Return the cleaned DataFrame and also optionally return a structured report (dictionary or DataFrame) that summarizes, per feature: method used, thresholds, number of outliers detected, number modified/removed, and any warnings. Include a short demonstration snippet showing how to call the improved functions on a dataset like insurance.csv and how to interpret the report.

Important: Do not use seaborn. Use matplotlib only if plotting is explicitly requested. If you include comparisons or inequalities in any code examples, keep them as standard Python operators (do not convert them to XML entities). Provide the final improved code as a clean, copy/paste-ready block.

After the AI produces an improved version, review the output carefully. Make sure you understand what changed, verify that the logic matches your intent, and test the new function on a dataset you know well before using it in a pipeline.

7.8 Summary and Guidance



In this chapter, you learned to automate data preparation by building reusable functions that transform raw data into analysis-ready datasets. The goal is a repeatable pipeline you can apply across many datasets with minimal modification.

A central design principle is separation of responsibilities: *data wrangling* may include dataset-specific, hard-coded fixes, but downstream cleaning steps

should be *fully generalizable* (driven by data types, distributions, and parameters rather than specific labels or values).

- Wrangling first: resolve source-specific inconsistencies (for example, standardizing region labels like "NE" vs. "Northeast").
- Generalizable functions next: apply consistent rules for dates/times, binning, transformations, missing data, and outliers across any dataset.

When you structure your pipeline this way, you gain improved code reuse, easier debugging, and more confidence that cleaning logic will behave consistently as new data arrives.

Next, let's review some of the key principles from each section.

Key Takeaways: Data Wrangling

Data wrangling is the dataset-specific work that makes data usable for analysis and modeling. Before using AI to generate or refactor code, you should understand the baseline steps well enough to evaluate correctness and defend design decisions.

Your baseline wrangling function focused on a few high-impact structural rules that remove low-value features and simplify downstream work:

1. Drop columns that are empty or exceed a missingness threshold (for example, 95% missing).
2. Drop columns that are mostly unique (for example, 95%+ unique), since these are often identifiers.
3. Drop columns with only a single repeated value.

You also reinforced reusable function design patterns: configurable thresholds, optional messaging/logging, and a consistent interface. When AI is used, treat the baseline function as a specification artifact—then test the AI-improved version across datasets to confirm it preserves intent and behaves transparently.

Key Takeaways: Dates and Times

Temporal fields often arrive as strings and need special handling before modeling. Raw dates are rarely good categorical features because they tend to have too many unique values and do not form stable, repeatable groups.

- Decompose date/time values into structured features (year, month, day, weekday, hour, minute).
- When relevant, prefer *elapsed-time* features (for example, days since last purchase) over raw timestamps.

Your baseline parsing function emphasized generalizable automation: consistent naming, optional dropping of raw columns, and *graceful failure* when columns are missing or values are malformed. AI-assisted improvements should add real-world robustness (format variance, invalid dates, time zones) while keeping logging configurable and avoiding unintended mutation of the input DataFrame.

Key Takeaways: Grouping Data into Bins

Binning (discretization) converts raw values into fewer, more meaningful groups to simplify analysis, improve interpretability, and reduce noise. The key distinction is between *categorical binning* and *numeric binning*, which solve different problems.

- Categorical binning manages high-cardinality and low-frequency groups (often using a diagnostic like the *5% rule*).
- Common actions include merging similar categories, grouping into *Other*, or removing affected records (guided by domain knowledge).
- Numeric binning creates ranges using fixed thresholds (business logic) or quantiles (distribution-based), trading precision for interpretability.

Your binning functions reinforced general-purpose design: apply rules based on data type, expose thresholds as parameters, and behave consistently across datasets—while keeping dataset-specific grouping decisions confined to early wrangling.

Key Takeaways: Math Transformations

Transformations improve numeric feature behavior (for example, skewness, heavy tails, or non-linear relationships) so data better match modeling assumptions—especially for linear methods that prefer stable variance and approximately linear relationships.

- Use standard, interpretable transformations (log, square root, Box-Cox, Yeo-Johnson, scaling/standardization) based on constraints and goals.
- Favor restraint: test a small “menu” of monotonic options and select what meaningfully improves distribution shape without over-engineering.
- Validate changes with simple diagnostics (skewness metrics and basic histograms) to ensure you improved behavior without creating artifacts.

Automation should remain broadly applicable: transformations can be parameterized and data-driven, while the decision of whether a feature should be transformed at all should be informed by domain context rather than hard-coded rules.

Key Takeaways: Missing Data

Missing values can distort conclusions and reduce model performance, so they must be handled deliberately. You learned three missingness patterns—**MCAR**, **MAR**, and **MNAR**—which influence whether imputation is reasonable.

Your automated workflow followed a practical decision sequence:

1. Drop the biggest offenders first using row/column missingness thresholds (while protecting the label column when applicable).
2. Then decide how to handle what remains by checking whether missingness appears outcome-biased.

You also practiced testing for outcome bias (numeric labels: compare means; categorical labels: compare proportions). When bias is suspected, use conservative, transparent handling; when bias is not evident, consider stronger imputers (chosen with scale in mind). The core requirement is explainability: expose thresholds, report what changed, and make decisions easy to justify.

Key Takeaways: Managing Outliers

Outliers are not automatically “bad data.” They may be errors, rare but valid cases, or important signals, so you must evaluate them in context and in relation to the modeling approach you plan to use.

You learned multiple detection strategies, each with different assumptions:

- **Empirical Rule (Z-score):** best when features are approximately normal; sensitive to skewness and extreme values.

- **Tukey’s IQR rule:** more robust for skewed data; uses medians and quartiles.
- **DBSCAN clustering:** identifies multivariate outliers by density (points not belonging to any dense region).

You also practiced multiple handling options—remove, cap/winsorize, transform, or keep and rely on robust models—recognizing that univariate methods operate feature-by-feature while clustering methods incorporate multivariate context. When automated, outlier handling should expose thresholds as parameters and log how many values were affected so decisions are not hidden.

Data Cleaning Pipelines

One question you still may have is whether we always follow the same path through all cleaning steps as they have been outlined in this chapter. The truth is there are some basic patterns that obvious to follow, but the exact path in detail often depends on the characteristics of the data set. The table below summarizes three possible use cases to illustrate this point.

Table 7.3
Automated Data Cleaning Pipelines: Alternative Paths Based on Dataset Characteristics

Step	Pipeline Stage	Path A: Mostly Clean / Small Dataset	Path B: Messy / Real-World Dataset	Path C: Large / High-Dimensional Dataset
------	----------------	--	--	--

Step	Pipeline Stage	Path A: Mostly Clean / Small Dataset	Path B: Messy / Real-World Dataset	Path C: Large / High-Dimensional Dataset
1	Initial Data Wrangling (Dataset-Specific)	Minimal renaming and type fixes	Extensive recoding, label standardization, known error fixes	Light wrangling only (avoid expensive custom logic)
2	Structural Cleanup	Drop empty, constant, and ID-like columns	Apply missingness and uniqueness thresholds aggressively	Apply strict thresholds to reduce dimensionality early
3	Dates and Times	Extract basic features (year, month, weekday)	Parse multiple formats and create elapsed-time features	Extract only high-signal time features to control feature growth
4	Categorical Binning	Apply light binning to rare categories	Use 5% rule and consolidate aggressively into “Other”	Bin early to control dummy-variable explosion
5	Numeric Binning (Optional)	Often skipped	Used for skewed or noisy numeric features	Used selectively to stabilize distributions
6	Math Transformations	Apply only if clear skewness exists	Menu-based skew correction (log, root, Yeo-Johnson)	Restrained transformations to avoid overfitting

Step	Pipeline Stage	Path A: Mostly Clean / Small Dataset	Path B: Messy / Real-World Dataset	Path C: Large / High-Dimensional Dataset
7	Missing Data Handling	Simple drops or median/mode fills	Bias checks followed by targeted imputation	Threshold-based drops and scalable imputers
8	Outlier Detection	Often skipped or handled via transformation	IQR or Z-score with replace/remove options	Multivariate methods (e.g., clustering) when feasible
9	Final Review	Spot-check distributions and counts	Review logs and summaries from each function	Verify row/feature counts and computational stability

In your career, you'll get practice with many different datasets which will help you gain confidence in the routes you take through data cleaning. Don't worry if you still feel apprehensive at this point about whether your data cleaning decisions will always be "right" or "wrong".

As you move into the assignment, focus on designing functions that preserve the chapter's core separation: dataset-specific wrangling first, then general-purpose, parameter-driven cleaning steps that are reusable across datasets, testable, and explainable.

7.9 Assignment

Complete the assignment below. If you do not see an assignment, then the content of this chapter is combined with the assignment for the next chapter.